

[TÊN TRƯỜNG]
[KHOA / BỘ MÔN]

BÁO CÁO ĐỒ ÁN
MÔN LẬP TRÌNH DI ĐỘNG
ỨNG DỤNG QUẢN LÝ
CHI TIÊU CÁ NHÂN

QLCT — Android | Jetpack Compose | MVVM | Room

Giảng viên hướng dẫn: [Bổ sung họ tên giảng viên]

Lớp: [Bổ sung tên lớp] • Nhóm: 4 thành viên

Bảng 1. Danh sách thành viên nhóm

STT	Họ và tên	Mã sinh viên
1	Nguyễn Như Vũ	25610008
2	Nguyễn Văn Vũ	25810053
3	Nguyễn Thiên Vọng	25810050
4	Nguyễn Văn Phước	25810035

Năm học: [Bổ sung]

Bản báo cáo theo mã nguồn ngày 06/10/2026

THÔNG TIN BÁO CÁO

Báo cáo trình bày ứng dụng QLCT đã triển khai trong repository QLCT_LTDD. Các mô tả nghiệp vụ, lớp, bảng dữ liệu và hình ảnh được đối chiếu với phiên bản Jetpack Compose, MVVM và Room version 2. Báo cáo Basic và đề cương cũ trong docs/ là tài liệu lịch sử, không phải mô tả kiến trúc hiện tại.

Các mục tên trường, khoa, lớp, giảng viên và năm học trên trang bìa chưa được cung cấp tại thời điểm biên soạn nên được để dưới dạng chỗ trống có nhãn. Nhóm cần hoàn thiện các mục này trước khi nộp. Phân công trong báo cáo là phương án đề xuất để tổ chức thuyết trình, không khẳng định người nào đã thực hiện một phần mã cụ thể.

Tóm tắt đồ án

Ứng dụng giúp người dùng ghi nhận khoản thu và khoản chi, xem lịch sử, chọn thời gian, lọc loại giao dịch và quan sát tổng quan tài chính. Dữ liệu được lưu cục bộ bằng Room trên SQLite. Giao diện sử dụng Compose Material 3 với ba trang Tổng quan, Giao dịch và Thống kê; biểu đồ vòng thể hiện phân bổ khoản chi theo danh mục, biểu đồ cột thể hiện tổng chi trong sáu tháng. Kiến trúc MVVM tách giao diện, trạng thái và truy cập dữ liệu để dễ kiểm thử và bảo trì.

Ứng dụng không cần tài khoản hoặc máy chủ để thực hiện các chức năng hiện có. Số tiền lưu dạng Long, luôn lớn hơn 0; loại INCOME hoặc EXPENSE quyết định ý nghĩa thu/chi. Có migration bảo toàn danh mục và giao dịch từ database cũ version 1. Các bằng chứng kiểm thử gồm unit test, instrumentation test, kiểm tra SQL migration trên máy tính và một số luồng thao tác trực tiếp trên máy ảo.

Từ khóa

Android; Kotlin; Jetpack Compose; MVVM; Room; SQLite; quản lý chi tiêu; kiểm thử; migration.

MỤC LỤC

THÔNG TIN BÁO CÁO	2
Tóm tắt đồ án	2
Từ khóa	2
DANH MỤC HÌNH	9
DANH MỤC BẢNG	10
DANH MỤC TỪ VIẾT TẮT	12
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	13
1.1. Lý do chọn đề tài	13
1.2. Mục tiêu của đồ án	13
1.3. Đối tượng sử dụng và phạm vi	13
1.4. Yêu cầu kỹ thuật của đề tài	14
1.4.1. Đề bài chính thức và phạm vi bắt buộc	14
1.4.2. Diễn giải giao diện 100% Jetpack Compose	15
1.4.3. Diễn giải kiến trúc MVVM	15
1.4.4. Diễn giải Room lưu giao dịch	16
1.4.5. Diễn giải biểu đồ chi tiêu theo tháng	16
1.5. Nhóm thực hiện và phương án phân công	17
1.6. Phương pháp thực hiện và cấu trúc báo cáo	17
CHƯƠNG 2. CƠ SỞ CÔNG NGHỆ	18
2.1. Android và Kotlin	18
2.2. Jetpack Compose và Material 3	18
2.3. Kiến trúc MVVM	18
2.4. Flow, StateFlow, lifecycle và lưu trạng thái	19
2.5. Room và SQLite	19
2.6. Long, ngày tháng và tính chính xác	20
2.7. Cấu hình thư viện của bản báo cáo	20

CHƯƠNG 3. PHÂN TÍCH YÊU CẦU VÀ USE CASE	21
3.1. Bài toán nghiệp vụ và quy tắc chung	21
3.2. Yêu cầu chức năng	21
3.3. Yêu cầu phi chức năng	22
3.4. Actor và sơ đồ use case	22
3.5. Đặc tả UC01 — Xem tổng quan	23
Luồng chính	23
Luồng thay thế và ngoại lệ	24
Điểm kiểm tra khi nghiệm thu	24
3.6. Đặc tả UC02 — Thêm giao dịch	24
Luồng chính	24
Luồng thay thế và ngoại lệ	25
Điểm kiểm tra khi nghiệm thu	25
3.7. Đặc tả UC03 — Xem lịch sử giao dịch	25
Luồng chính	25
Luồng thay thế và ngoại lệ	26
Điểm kiểm tra khi nghiệm thu	26
3.8. Đặc tả UC04 — Sửa giao dịch	26
Luồng chính	26
Luồng thay thế và ngoại lệ	26
Điểm kiểm tra khi nghiệm thu	27
3.9. Đặc tả UC05 — Xóa giao dịch	27
Luồng chính	27
Luồng thay thế và ngoại lệ	27
Điểm kiểm tra khi nghiệm thu	28
3.10. Đặc tả UC06 — Chọn khoảng thời gian	28
Luồng chính	28

Luồng thay thế và ngoại lệ	28
Điểm kiểm tra khi nghiệm thu	28
3.11. Đặc tả UC07 — Loại giao dịch	29
Luồng chính	29
Luồng thay thế và ngoại lệ	29
Điểm kiểm tra khi nghiệm thu	29
3.12. Đặc tả UC08 — Xem phân bổ chi theo danh mục	29
Luồng chính	30
Luồng thay thế và ngoại lệ	30
Điểm kiểm tra khi nghiệm thu	30
3.13. Đặc tả UC09 — Xem chi tiêu theo tháng	30
Luồng chính	30
Luồng thay thế và ngoại lệ	31
Điểm kiểm tra khi nghiệm thu	31
3.14. Ma trận truy vết yêu cầu	31
CHƯƠNG 4. THIẾT KẾ KIẾN TRÚC VÀ CƠ SỞ DỮ LIỆU	32
4.1. Kiến trúc tổng thể	32
4.2. Trách nhiệm của từng thành phần	32
4.3. Thiết kế trạng thái giao diện	34
4.4. Mô hình quan hệ dữ liệu	35
4.4.1. Từ điển bảng categories	35
4.4.2. Từ điển bảng transactions	36
4.4.3. Khóa và chỉ mục	36
4.5. Phân biệt ràng buộc database và validate ứng dụng	36
4.6. DDL và truy vấn theo schema xuất từ KSP	37
4.6.1. Tạo bảng categories	37
4.6.2. Tạo bảng transactions	37

4.6.3. Truy vấn đọc danh sách	37
4.6.4. Ghi dữ liệu và truy vấn tham số	37
4.7. Dữ liệu khởi tạo và ví dụ tính toán	38
4.7.1. Diễn giải INSERT danh mục trong seed.sql	39
4.7.2. Diễn giải INSERT giao dịch và tổng mẫu	39
4.7.3. Ngày khởi tạo và quan hệ với chart tháng	39
4.8. Thiết kế migration version 1 sang version 2	40
4.9. Bảo vệ dữ liệu và các giới hạn lưu trữ	41
CHƯƠNG 5. TRIỂN KHAI CHỨC NĂNG VÀ GIAO DIỆN	42
5.1. Thiết kế điều hướng và ngôn ngữ giao diện	42
5.2. Triển khai thêm giao dịch	43
5.3. Triển khai sửa giao dịch	44
5.4. Triển khai xóa có xác nhận	45
5.5. Validate số tiền và bảo vệ đầu vào	46
5.6. Tổng hợp, lọc và các công thức thống kê	48
5.6.1. Xác định period và danh sách visible	48
5.6.2. Tổng thu, tổng chi và chênh lệch	48
5.6.3. Biểu đồ vòng	49
5.6.4. Biểu đồ cột sáu tháng	49
5.7. Sequence ghi dữ liệu và cập nhật UI	50
5.8. Xử lý trạng thái rỗng, lỗi và thao tác đồng thời	50
5.9. Ảnh chụp giao diện thực tế	51
5.9.1. Tổng quan và lịch sử	52
5.9.2. Thống kê và biểu mẫu thêm	53
5.9.3. Chọn danh mục và ngày	54
5.9.4. Nhập liệu và lỗi số tiền	55
CHƯƠNG 6. KIỂM THỬ VÀ HƯỚNG DẪN VẬN HÀNH	56

6.1. Mục tiêu, phương pháp và môi trường	56
6.2. Kết quả kiểm thử tự động	56
6.3. Kiểm thử giao diện trực tiếp và ma trận nghiệm thu	57
6.4. Cách tái thực hiện các kiểm tra	58
6.5. Cài đặt và sử dụng ứng dụng	59
6.6. Lưu ý nhập liệu trên máy ảo	59
CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	60
7.1. Kết quả đạt được	60
7.2. Hạn chế hiện tại	60
7.3. Kế hoạch phát triển theo mức ưu tiên	60
TÀI LIỆU THAM KHẢO	62
PHỤ LỤC A. ÁNH XẠ MÃ NGUỒN VÀ TÀI LIỆU	63
PHỤ LỤC B. KỊCH BẢN THUYẾT TRÌNH VÀ DEMO	64
PHỤ LỤC C. CÂU HỎI BẢO VỆ VÀ GIẢI THÍCH KỸ THUẬT	65
C.1. Vì sao tiền phải dương trong cả thu và chi?	65
C.2. Vì sao dùng Long thay vì Double?	65
C.3. Room khác SQLite như thế nào?	65
C.4. MVVM phân công trách nhiệm ra sao?	65
C.5. Tại sao Flow làm giao diện cập nhật?	65
C.6. Dữ liệu có mất khi cài bản APK mới không?	65
C.7. Tại sao không dùng destructive migration?	65
C.8. Tại sao chart không đôi khi lọc Thu?	66
C.9. Tổng phần trăm donut vì sao có thể là 98%?	66
C.10. Foreign key và index có ý nghĩa gì?	66
C.11. Có bảo vệ số âm ngay trong SQL không?	66
C.12. Đã kiểm thử mọi màn hình chưa?	66
PHỤ LỤC D. CHECKLIST TRƯỚC KHI NỘP	67

DANH MỤC HÌNH

- Hình 1. Use case tổng quát của bản Compose/MVVM/Room 23
- Hình 2. Kiến trúc MVVM và hai chiều sự kiện – trạng thái 32
- Hình 3. Sơ đồ lớp chính và quan hệ phụ thuộc trong mã hiện tại 33
- Hình 4. ERD theo schema Room version 2 35
- Hình 5. Luồng mở database mới, đã có version 2 hoặc migration version 1 40
- Hình 6. Điều hướng ba tab và các hộp thoại của QLCT 42
- Hình 7. Flow thêm giao dịch với nhánh dữ liệu sai và ghi thất bại 43
- Hình 8. Flow sửa giao dịch và kiểm tra bản ghi còn tồn tại 44
- Hình 9. Flow xóa với nhánh hủy và lỗi ghi 45
- Hình 10. Flow validate số tiền từ chuỗi nhập tới Long hợp lệ 46
- Hình 11. Flow tính period, visible, donut và chart 48
- Hình 12. Sequence save từ View tới Room và chiều cập nhật Flow 50
- Hình 13. Màn hình Tổng quan với tổng thu/chi và hai biểu đồ 52
- Hình 14. Màn hình Giao dịch, thứ tự ngày/ID và các thao tác 52
- Hình 15. Thống kê với số tiền theo danh mục 53
- Hình 16. Biểu mẫu thêm giao dịch bằng Compose 53
- Hình 17. DropdownMenu chọn một trong 7 danh mục 54
- Hình 18. DatePickerDialog chọn ngày giao dịch 54
- Hình 19. Bàn phím số và ô nhập giao dịch trên máy ảo 55
- Hình 20. Lỗi inline khi nhập số tiền âm, biểu mẫu được giữ 55

DANH MỤC BẢNG

Bảng 1. Danh sách thành viên nhóm	1
Bảng 2. Từ viết tắt và ý nghĩa	12
Bảng 3. Phạm vi triển khai và phần chưa triển khai	14
Bảng 4. Đối chiếu yêu cầu chính thức với thành phần hiện thực	14
Bảng 5. Phân công đề xuất để hoàn thiện và bảo vệ đồ án	17
Bảng 6. Công nghệ và phiên bản từ cấu hình project	20
Bảng 7. Các quy tắc nghiệp vụ đang áp dụng	21
Bảng 8. Danh sách yêu cầu chức năng	21
Bảng 9. Thông tin use case UC01	23
Bảng 10. Thông tin use case UC02	24
Bảng 11. Thông tin use case UC03	25
Bảng 12. Thông tin use case UC04	26
Bảng 13. Thông tin use case UC05	27
Bảng 14. Thông tin use case UC06	28
Bảng 15. Thông tin use case UC07	29
Bảng 16. Thông tin use case UC08	29
Bảng 17. Thông tin use case UC09	30
Bảng 18. Truy vết yêu cầu – use case – mã nguồn – kiểm tra	31
Bảng 19. Trách nhiệm lớp và thành phần giao diện	32
Bảng 20. Các trường ExpenseUiState	34
Bảng 21. Các trạng thái thao tác và vị trí lưu	34
Bảng 22. Từ điển dữ liệu categories	35
Bảng 23. Từ điển dữ liệu transactions	36
Bảng 24. Các chỉ mục và hành động khóa ngoại	36
Bảng 25. Bẫy danh mục có sẵn	38
Bảng 26. Tám giao dịch mẫu khi tạo database lần đầu	38
Bảng 27. Thành phần giao diện và tương tác	42
Bảng 28. Thông báo validate theo dữ liệu đầu vào	46
Bảng 29. Môi trường kiểm thử và giới hạn bằng chứng	56
Bảng 30. Các unit test đã đạt	56
Bảng 31. Các kiểm tra trong instrumentation test	57
Bảng 32. Ma trận chức năng và tình trạng xác minh	58

Bảng 33. Đề xuất phát triển sau phạm vi hiện tại 60

Bảng 34. Tập phục vụ đọc và bảo vệ đồ án 63

Bảng 35. Kịch bản demo có thể thực hiện 64

DANH MỤC TỪ VIẾT TẮT

Bảng 2. Từ viết tắt và ý nghĩa

Thuật ngữ	Giải thích
UI	User Interface — giao diện người dùng
MVVM	Model – View – ViewModel
DAO	Data Access Object — lớp/interface truy cập dữ liệu
CRUD	Create, Read, Update, Delete — thêm, đọc, sửa, xóa
ERD	Entity Relationship Diagram — sơ đồ quan hệ thực thể
PK / FK	Primary Key / Foreign Key — khóa chính / khóa ngoại
VND	Đồng Việt Nam; sử dụng số nguyên trong ứng dụng
Flow / StateFlow	Luồng dữ liệu bất đồng bộ / luồng giữ trạng thái mới nhất
KSP	Kotlin Symbol Processing — sinh mã Room
AVD	Android Virtual Device — cấu hình máy ảo Android
SDK / API	Software Development Kit / Application Programming Interface
IME	Input Method Editor — phương thức nhập/bàn phím Android

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong sinh hoạt và học tập, các khoản chi nhỏ thường phát sinh nhiều lần trong ngày. Nếu chỉ ghi nhớ hoặc ghi chép rời rạc, người dùng khó biết tổng chi của tháng, khó đối chiếu với thu nhập và khó nhận ra nhóm chi nào chiếm tỷ trọng lớn. Một ứng dụng di động có thể hỗ trợ ghi nhận ngay tại thời điểm phát sinh, tổng hợp lại theo khoảng thời gian và trình bày kết quả bằng các hình thức dễ quan sát.

Đề tài “Ứng dụng quản lý chi tiêu cá nhân” được lựa chọn vì có nghiệp vụ gần gũi, đồng thời tạo điều kiện vận dụng các nội dung quan trọng của môn Lập trình di động: xây dựng giao diện, xử lý sự kiện, quản lý trạng thái, lưu trữ dữ liệu và kiểm thử. Phạm vi được giới hạn vào quản lý giao dịch cục bộ để bảo đảm sản phẩm có thể chạy và trình diễn mà không phụ thuộc tài khoản, mạng hoặc dịch vụ bên ngoài.

Tên gọi QLCT là viết tắt của Quản lý chi tiêu. Ứng dụng là công cụ ghi chép và thống kê dữ liệu do người dùng nhập; không kết nối tài khoản ngân hàng, không tự động lấy giao dịch và không đưa ra quyết định tài chính thay người dùng.

1.2. Mục tiêu của đồ án

- Xây dựng ứng dụng Android có thể thêm, xem, sửa và xóa các giao dịch thu/chi, lưu được dữ liệu sau khi đóng và mở lại.
- Triển khai giao diện hoàn toàn bằng Jetpack Compose, tổ chức mã theo MVVM và dùng Room làm lớp lưu trữ giao dịch.
- Tính tổng thu, tổng chi, chênh lệch theo tháng được chọn hoặc toàn bộ thời gian; cung cấp bộ lọc loại giao dịch.
- Vẽ biểu đồ chi tiêu theo tháng và phân bổ khoản chi theo danh mục từ dữ liệu thực tế, xử lý rõ trường hợp không có dữ liệu.
- Kiểm tra dữ liệu đầu vào, đặc biệt là số tiền dương, đồng thời bảo toàn dữ liệu khi chuyển từ SQLiteOpenHelper phiên bản cũ sang Room.
- Hoàn thiện tài liệu phân tích, thiết kế và kiểm thử để giải thích được cách hệ thống hoạt động, không chỉ trình bày ảnh giao diện.

1.3. Đối tượng sử dụng và phạm vi

Đối tượng sử dụng là người dùng cá nhân cần ghi lại các khoản thu và chi bằng VND trên một thiết bị Android. Mỗi bản cài đặt có một database riêng; hệ thống không có

tài khoản và không tách nhiều hồ sơ người dùng. Trong mô hình use case chỉ có một actor “Người dùng”.

Bảng 3. Phạm vi triển khai và phần chưa triển khai

Nhóm	Nội dung
Đã triển khai	CRUD giao dịch; 7 danh mục cố định; chọn ngày; ghi chú; chọn tháng; lọc Thu/Chi; tổng quan; biểu đồ vòng; biểu đồ cột 6 tháng; migration v1→v2.
Chưa triển khai	Đăng nhập; đồng bộ cloud; ngân sách; nhắc nhở; xuất/nhập dữ liệu; CRUD danh mục; tìm kiếm từ khóa; lọc riêng theo danh mục.
Giới hạn môi trường	Android API 24 trở lên; compileSdk/targetSdk 37; kiểm tra chạy thực tế hiện có trên Pixel 6 máy ảo Android 15/API 35.
Giới hạn dữ liệu	VND dạng số nguyên; không hỗ trợ đa tiền tệ; các giao dịch được nạp toàn bộ và tổng hợp trong bộ nhớ.

1.4. Yêu cầu kỹ thuật của đề tài

Bảng 4. Đối chiếu yêu cầu chính thức với thành phần hiện thực

Yêu cầu	Thành phần đáp ứng	Bằng chứng
100% Jetpack Compose	setContent, ExpenseScreen, TransactionEditor, DashboardComponents	Không có res/layout trong giao diện hiện tại; DatePicker và các dialog là Compose.
Kiến trúc MVVM	ExpenseViewModel, ExpenseRepository, ExpenseDao, ExpenseUiState	Giao diện gửi sự kiện vào ViewModel; trạng thái quan sát bằng StateFlow.
Room lưu giao dịch	ExpenseDatabase, Category, TransactionEntity, ExpenseDao	@Database version=2; @Entity; @Dao; KSP xuất schema 2.json.
Biểu đồ chi tiêu tháng	summarize(), MonthlyExpense, SpendingChart	6 cột tương ứng 6 tháng kết thúc tại tháng được chọn; chi cộng khoản EXPENSE.

Việc còn các file XML như AndroidManifest.xml, theme hoặc drawable không vi phạm yêu cầu không dùng XML layout. Các file này phục vụ cấu hình ứng dụng và tài nguyên, còn các màn hình nghiệp vụ được dựng bằng Compose.

1.4.1. Đề bài chính thức và phạm vi bắt buộc

Theo bảng yêu cầu được cung cấp, nhóm 1 có chủ đề “Jetpack Compose”, đề tài đồ án cuối kỳ “Ứng dụng Quản lý chi tiêu cá nhân”. Yêu cầu kỹ thuật bắt buộc được ghi nguyên văn: “Giao diện 100% Jetpack Compose (không XML); kiến trúc MVVM; Room lưu giao dịch; biểu đồ chi tiêu theo tháng.” Đây là căn cứ chính của phần triển khai và đánh giá kết quả báo cáo.

Bốn yêu cầu phải được đáp ứng đồng thời. Thêm/sửa/xóa, lọc và tổng thu/chi là nghiệp vụ phục vụ đề tài. Validate tiền là yêu cầu bổ sung đã thống nhất. Biểu đồ vòng danh mục hỗ trợ phân tích; biểu đồ cột tháng đáp ứng trực tiếp yêu cầu bắt buộc. Đăng nhập, cloud, ngân sách và export từ ứng dụng chưa có trong ảnh đề bài và chưa được tính là tính năng đã triển khai.

1.4.2. Diễn giải giao diện 100% Jetpack Compose

Yêu cầu quy định công nghệ dựng toàn bộ màn hình nghiệp vụ. Tổng quan, lịch sử, thống kê, biểu mẫu thêm/sửa, xác nhận xóa, bộ chọn ngày, ô tiền, chip lọc và điều hướng được dựng bằng hàm Composable trong Kotlin, không bố trí bằng XML layout.

MainActivity gọi setContent để khởi tạo giao diện. ExpenseScreen điều phối ba tab; TransactionEditor tạo biểu mẫu bằng Compose Material 3. Các thành phần nhận giá trị từ state và phát sự kiện khi người dùng thao tác. Khi state đổi, Compose cập nhật phần giao diện phụ thuộc state; không dùng findViewById để sửa từng nhãn.

Ví dụ nhập 0 và nhấn Lưu: validator trả lỗi, ô tiền chuyển trạng thái lỗi và supportingText hiện thông báo. Sửa thành 50000 làm kiểm tra được tính lại. Loại, danh mục, ngày và ghi chú cũng được giữ bằng state.

XML manifest/theme/drawable là cấu hình và tài nguyên Android, không phải layout màn hình. Cách chứng minh là kiểm tra setContent, @Composable và không có XML layout được dùng để dựng màn hình. Demo mở ba tab, editor, danh mục và DatePicker để trình bày cả màn hình chính lẫn dialog; ảnh chương 5 là giao diện chạy thật.

1.4.3. Diễn giải kiến trúc MVVM

MVVM tách View, ViewModel và Model để giao diện không tự viết SQL và lớp dữ liệu không điều khiển nút/dialog. View là các Composable, nhận thao tác và hiển thị ExpenseUiState, busy và lỗi. ExpenseViewModel giữ tháng/bộ lọc bằng SavedStateHandle, combine Flow, tính summarize và điều phối save/delete trong viewModelScope.

Model gồm entity, ExpenseRepository, ExpenseDao và ExpenseDatabase. Repository kiểm tra nghiệp vụ; DAO định nghĩa truy vấn; Room lưu SQLite. Các lớp này không dựng giao diện và không tự đóng biểu mẫu.

Luồng thêm: View nhận dữ liệu hợp lệ → ViewModel.save bật busy → Repository kiểm tra tiền/loại/ngày/danh mục → DAO insert Room. Ghi thành công làm completedActions tăng để View đóng biểu mẫu. Room phát Flow mới, ViewModel

tổng hợp và Compose hiện tổng/biểu đồ mới. Hoàn tất ghi và phát Flow là các sự kiện bất đồng bộ.

Luồng đổi bộ lọc: View gửi lựa chọn → ViewModel cập nhật SavedStateHandle → combine tạo UiState → Compose đổi danh sách. Không ghi database. Chứng minh MVVM bằng trách nhiệm và chiều phụ thuộc của lớp qua một thao tác cụ thể, liên hệ sơ đồ kiến trúc/class/sequence; không chỉ nhìn tên ViewModel.

1.4.4. Diễn giải Room lưu giao dịch

Dữ liệu phải được lưu bền vững trên thiết bị, không chỉ ở danh sách tạm. Room là lớp truy cập SQLite qua entity, DAO và quản lý phiên bản database. Các chức năng hiện tại không cần server.

ExpenseDatabase khai báo qlct.db version 2; Category ánh xạ categories, TransactionEntity ánh xạ transactions gồm ID, loại, tiền, category_id, ngày và note. category_id tham chiếu categories.id. ID giao dịch tự tăng, ID danh mục được xác định bởi dữ liệu khởi tạo.

ExpenseDao dùng @Insert, @Update và delete; truy vấn Flow JOIN danh mục lấy tên, sắp date DESC rồi id DESC. Insert trả ID mới; update/delete trả số dòng để Repository kiểm tra tác động đúng bản ghi. Room bảo vệ PK/FK/NOT NULL/index; tiền dương được kiểm tra tại UI/Repository vì schema v2 chưa có CHECK cho amount/type.

Có thể demo bằng cách thêm dòng đánh dấu, đóng/mở lại và đổi chiều dữ liệu. Instrumentation đã kiểm tra CRUD, mở lại và migration v1→v2. Cài đề APK giữ dữ liệu; gỡ app/clear storage xóa dữ liệu riêng. seed.sql khởi tạo một lần; nhấn Lưu về sau đi qua Repository/DAO chứ không sửa seed.

1.4.5. Diễn giải biểu đồ chi tiêu theo tháng

Biểu đồ lấy giao dịch đã lưu, chỉ chọn EXPENSE và cộng amount theo tháng của date. Mỗi cột có tháng rõ ràng; tháng rỗng nhận 0; INCOME không cộng vào cột chi.

summarize tạo sáu tháng M-5 đến M, M là tháng được chọn. Ví dụ M=10/2026, chart là 05/2026 đến 10/2026. MonthlyExpense giữ tháng/tổng chi và SpendingChart vẽ cột. Chiều cao chuẩn hóa theo tổng lớn nhất với mẫu số ít nhất 1 tránh chia 0. Tiền giữ Long; kiểu thực phục vụ tỷ lệ vẽ.

Với database mới chưa chỉnh sửa, sáu khoản chi mẫu 45.000, 70.000, 150.000, 250.000, 800.000 và 30.000 tạo cột tháng khởi tạo 1.345.000 VND. Hai khoản thu 3.000.000 và 1.200.000 không vào chart. Các tháng trước nhận 0 nếu chưa có giao dịch; không tự sinh số liệu.

Thêm/sửa/xóa chi làm Flow phát dữ liệu và chart tính lại. Lọc Thu/Chi chi tác động visible; chart luôn dùng chi từ toàn bộ rows trong sáu tháng. Toàn bộ thay đổi kỳ của danh sách/tổng/donut, nhưng chart vẫn là sáu tháng kết thúc tại M.

Nghiệm thu bằng các khoản chi ở hai tháng và một khoản thu: cộng tay từng tháng, đối chiếu cột; kiểm tra tháng rỗng và 12→1. Unit test đã kiểm tra cửa sổ sáu tháng, bỏ thu và độc lập lọc loại. Donut là phần hỗ trợ, không thay thế chart tháng bắt buộc.

1.5. Nhóm thực hiện và phương án phân công

Bảng 5. Phân công đề xuất để hoàn thiện và bảo vệ đồ án

Thành viên / MSSV	Phần đề xuất phụ trách	Sản phẩm chuẩn bị
Nguyễn Như Vũ 25610008	Mô hình dữ liệu, Room, DAO, migration	Giải thích ERD; schema; truy vấn; bảo toàn dữ liệu và test database.
Nguyễn Văn Vũ 25810053	Giao diện Compose và biểu mẫu giao dịch	Trình diễn thêm/sửa/xóa; xử lý bản phím, danh mục, DatePicker và validate.
Nguyễn Thiên Vọng 25810050	ViewModel, lọc và thống kê	Giải thích StateFlow, tổng quan, biểu đồ vòng/cột và các công thức tổng hợp.
Nguyễn Văn Phước 25810035	Tích hợp, kiểm thử, báo cáo và trình diễn	Tổng hợp bằng chứng kiểm thử; kịch bản demo; định dạng Word và chuẩn bị slide.

Bảng trên là đề xuất phân công, cần nhóm xác nhận theo phần việc thực tế. Báo cáo không gán tỷ lệ đóng góp, thời gian làm việc hoặc thành tích cá nhân khi chưa có dữ liệu xác nhận.

1.6. Phương pháp thực hiện và cấu trúc báo cáo

Phương pháp thực hiện gồm xác định phạm vi, phân tích các thao tác của người dùng, thiết kế dữ liệu và trạng thái, xây dựng giao diện, kiểm tra nghiệp vụ, chạy máy ảo và đối chiếu tài liệu với mã. Các sơ đồ trong báo cáo phản ánh cấu trúc hiện tại, không lấy các chức năng ngân sách hoặc quản lý danh mục ở đề cương cũ làm kết quả đã hoàn thành.

Báo cáo gồm bảy chương: giới thiệu; công nghệ; phân tích yêu cầu và use case; thiết kế kiến trúc và dữ liệu; triển khai chức năng và giao diện; kiểm thử và hướng dẫn vận hành; kết luận. Phụ lục cung cấp cấu trúc mã, truy vấn, kịch bản demo và câu hỏi chuẩn bị bảo vệ.

CHƯƠNG 2. CƠ SỞ CÔNG NGHỆ

2.1. Android và Kotlin

Android là nền tảng chạy ứng dụng. Kotlin được sử dụng cho Activity, lớp dữ liệu, repository, ViewModel và toàn bộ hàm giao diện Compose. data class giúp biểu diễn các đối tượng như TransactionEntity, TransactionRow và ExpenseUiState với cấu trúc rõ ràng. Giá trị null được dùng trong các kết quả validate để biểu thị không có thông báo lỗi.

Project sử dụng Android Gradle Plugin 9.4.1 với hỗ trợ Kotlin tích hợp. Compose compiler plugin dùng phiên bản 2.2.10; KSP 2.3.6 sinh mã cho Room. Java sourceCompatibility và targetCompatibility là 11, trong khi JVM chạy Gradle được cấu hình bằng toolchain của project. Không đồng nhất hai khái niệm mức bytecode đích và JDK chạy hệ thống build.

2.2. Jetpack Compose và Material 3

Compose là công cụ xây dựng giao diện Android theo cách khai báo [1]. Trong QLCT, một hàm @Composable nhận dữ liệu và mô tả thành phần cần hiển thị. Khi state thay đổi, Compose tái kết hợp các phần liên quan để giao diện phản ánh dữ liệu mới. Không tìm View bằng findViewById và không tạo XML layout cho các trang nghiệp vụ.

Material 3 cung cấp Scaffold, Surface, AlertDialog, OutlinedTextField, FilterChip, DatePickerDialog và các nút. Các thành phần này được kết hợp với Row, Column, Box và LazyColumn để tạo giao diện tông tím. RoundedCornerShape tạo thẻ bo góc; Brush tạo gradient ở phần đầu; Canvas dùng để vẽ biểu đồ và icon bằng mã.

Giao diện có ba tab do biến tab lưu bằng rememberSaveable điều khiển. Đây là điều hướng bằng trạng thái trong một Activity; project hiện tại không dùng NavController. Các biểu mẫu thêm/sửa, menu danh mục, chọn ngày và xác nhận xóa là lớp giao diện tạm thời phủ lên trang đang xem.

2.3. Kiến trúc MVVM

MVVM chia trách nhiệm giữa View, ViewModel và Model. View tiếp nhận thao tác và hiển thị; ViewModel giữ trạng thái, xử lý các lệnh và kết nối dữ liệu với màn hình; Model bao gồm entity, repository, DAO và database. Cách tách trách nhiệm này phù hợp với định hướng phân lớp của Android [2] và đặc điểm ViewModel [3].

ExpenseScreen không tự chạy SQL. Khi nhấn Lưu, giao diện tạo TransactionEntity và gọi model.save(). ExpenseViewModel thực hiện tác vụ trong viewModelScope, cập nhật trạng thái busy và actionError. ExpenseRepository kiểm tra quy tắc nghiệp vụ trước khi gọi DAO. Room chịu trách nhiệm thực thi truy cập SQLite và phát lại dữ liệu qua Flow.

2.4. Flow, StateFlow, lifecycle và lưu trạng thái

DAO trả Flow cho danh mục và danh sách giao dịch. ViewModel dùng combine để kết hợp hai luồng dữ liệu với ba tham số month, monthOnly và type. Hàm summarize tạo kết quả tổng hợp, sau đó stateIn chuyển kết quả thành StateFlow cung cấp ExpenseUiState cho giao diện. SharingStarted.WhileSubscribed(5_000) duy trì chia sẻ trong một khoảng ngắn sau khi không còn subscriber.

ExpenseScreen dùng collectAsStateWithLifecycle để quan sát state trong vòng đời màn hình. SavedStateHandle lưu tháng và các bộ lọc; rememberSaveable lưu các trạng thái của biểu mẫu như loại, số tiền, ngày, ghi chú và ID đang chỉnh sửa. Cơ chế này hỗ trợ tái tạo giao diện, nhưng không nên diễn đạt thành bảo đảm mọi tình huống tắt cưỡng bức hoặc mọi giao dịch đang ghi đều được khôi phục; các tình huống đó cần kiểm thử riêng.

busy ngăn lặp thao tác ghi; actionError mang thông báo lỗi; completedActions tăng sau khi thao tác thành công. Giao diện dùng LaunchedEffect quan sát completedActions để đóng hộp thoại. Dữ liệu thực tế không được cập nhật bằng cách sửa danh sách hiển thị thủ công; danh sách mới xuất phát từ Flow của Room.

2.5. Room và SQLite

Room là thư viện cung cấp lớp truy cập dữ liệu trên SQLite [4]. Entity định nghĩa bảng, DAO định nghĩa thao tác truy cập, RoomDatabase cung cấp database và DAO. SQLite vẫn là hệ quản trị lưu trữ thực tế trên thiết bị. Người dùng không cần cài SQLite server hoặc dịch vụ database riêng.

Project dùng Room 2.8.4, xuất schema version 2 để lưu cấu trúc có thể kiểm tra. Database tên qlct.db nằm trong vùng dữ liệu riêng của ứng dụng. Room được mở bằng singleton để hạn chế tạo nhiều đối tượng quản lý cùng file. Các thao tác ghi DAO là suspend và không dùng allowMainThreadQueries().

Migration là quá trình nâng cấp cấu trúc database mà vẫn giữ dữ liệu [5]. MIGRATION_1_2 chuyển bảng tạo bởi SQLiteOpenHelper cũ sang schema mà Room kỳ vọng, đặc biệt là nullability của khóa chính và định nghĩa chỉ mục. Không dùng

`fallbackToDestructiveMigration()`, vì vậy một phiên bản chưa có đường migration phù hợp sẽ không tự xóa database để mở lại.

2.6. Long, ngày tháng và tính chính xác

Số tiền VND được lưu bằng Long để giữ chính xác giá trị nguyên và tránh sai số kiểu dấu phẩy động trong phép cộng. Mức nhập hợp lệ là 1 đến 999.999.999.999. Thu và chi đều lưu số dương; field type quyết định cách tính và dấu khi hiển thị. Chênh lệch tổng thu trừ tổng chi được phép âm vì đó là kết quả tổng hợp, không phải số tiền của một giao dịch.

Ngày lưu theo chuỗi ISO yyyy-MM-dd và được kiểm tra bằng `LocalDate.parse()`. Tháng được biểu diễn bằng `YearMonth`. `DatePicker` truyền mốc thời gian UTC rồi chuyển thành `LocalDate` theo UTC để không làm lệch ngày do đổi múi giờ. Dữ liệu mẫu được SQLite tạo ngày theo localtime của thiết bị khi database được tạo lần đầu.

2.7. Cấu hình thư viện của bản báo cáo

Bảng 6. Công nghệ và phiên bản từ cấu hình project

Thành phần	Phiên bản / giá trị	Vai trò
Android Gradle Plugin	9.4.1	Build ứng dụng Android; Kotlin tích hợp
Kotlin Compose compiler	2.2.10	Biên dịch các hàm Compose
Compose BOM	2025.08.00	Đồng bộ các thư viện Compose
Activity Compose	1.11.0	<code>ComponentActivity.setContent</code>
Lifecycle	2.9.4	<code>ViewModel</code> , <code>StateFlow</code> và quan sát lifecycle
Room	2.8.4	Entity, DAO, database và test
KSP	2.3.6	Sinh mã Room
JUnit	4.13.2	Unit test
minSdk / targetSdk / compileSdk	24 / 37 / 37	Mức Android tối thiểu và API biên dịch
Máy ảo kiểm tra	Pixel 6, API 35, x86_64, RAM 4 GB	Trình diễn giao diện và chạy instrumentation

CHƯƠNG 3. PHÂN TÍCH YÊU CẦU VÀ USE CASE

3.1. Bài toán nghiệp vụ và quy tắc chung

Người dùng ghi nhận một giao dịch bằng sáu thông tin: loại thu/chi, số tiền, danh mục, ngày và ghi chú, cùng ID do hệ thống quản lý. Một giao dịch thuộc một danh mục trong danh sách có sẵn. Thao tác chỉnh sửa thay đổi chính bản ghi đã chọn; thao tác xóa loại bỏ bản ghi sau khi được xác nhận. Hệ thống phải tính lại thông tin tổng hợp khi dữ liệu thay đổi.

Bảng 7. Các quy tắc nghiệp vụ đang áp dụng

Mã	Quy tắc	Thành phần chịu trách nhiệm
BR01	type chỉ INCOME hoặc EXPENSE	Biểu mẫu và ExpenseRepository
BR02	amount là số nguyên từ 1 đến 999.999.999.999	AmountValidator dùng chung; repository kiểm tra lại
BR03	categoryId phải là danh mục tồn tại	Chọn trong biểu mẫu; FK của database bảo vệ tham chiếu
BR04	Ngày lưu theo yyyy-MM-dd; không cấm ngày tương lai	DatePicker và LocalDate.parse trong repository
BR05	Ghi chú không bắt buộc; trim trước khi lưu	ExpenseRepository; schema note NOT NULL DEFAULT "
BR06	Thu/chi đều lưu tiền dương; tổng chênh lệch có thể âm	TransactionEntity và summarize
BR07	Tổng quan và donut không đổi theo bộ lọc Thu/Chi	summarize tính period trước visible
BR08	Biểu đồ cột gồm 6 tháng, độc lập monthOnly và type	summarize tạo chart từ toàn bộ rows
BR09	Chi đóng dialog sau thao tác ghi thành công	completedActions và LaunchedEffect
BR10	Dữ liệu mẫu chỉ xuất hiện khi tạo database mới	Room callback.onCreate

3.2. Yêu cầu chức năng

Bảng 8. Danh sách yêu cầu chức năng

Mã	Yêu cầu	Tiêu chí quan sát
FR01	Xem tổng quan thu/chi	Tổng thu, chi và chênh lệch phản ánh thời gian đang chọn.
FR02	Thêm khoản thu hoặc chi	Nhập hợp lệ, lưu vào Room và xuất hiện trong danh sách.
FR03	Xem danh sách và chi tiết ngắn	Sắp xếp ngày giảm dần, ID giảm dần; hiển thị danh mục, tiền, ngày, ghi chú.
FR04	Sửa một giao dịch	Biểu mẫu nạp giá trị cũ; update đúng ID; tổng hợp phản ánh dữ liệu mới.

Mã	Yêu cầu	Tiêu chí quan sát
FR05	Xóa có xác nhận	Hủy không xóa; xác nhận mới gọi DAO.delete().
FR06	Chọn tháng hoặc toàn bộ thời gian	Nút trước/sau đổi tháng; nút Theo tháng/Toàn bộ đổi phạm vi.
FR07	Lọc Tất cả/Thu/Chi	Tab Giao dịch dùng FilterChip; danh sách phù hợp loại đã chọn.
FR08	Xem phân bổ chi theo danh mục	Chỉ dùng khoản chi trong period; donut và chú giải tương ứng.
FR09	Xem chi tiêu theo tháng	6 tháng liên tiếp; tháng không có khoản chi nhận 0.
FR10	Xử lý lỗi và dữ liệu rỗng	Báo lỗi nhập; giữ form khi ghi thất bại; có Thử lại khi tải lỗi.

3.3. Yêu cầu phi chức năng

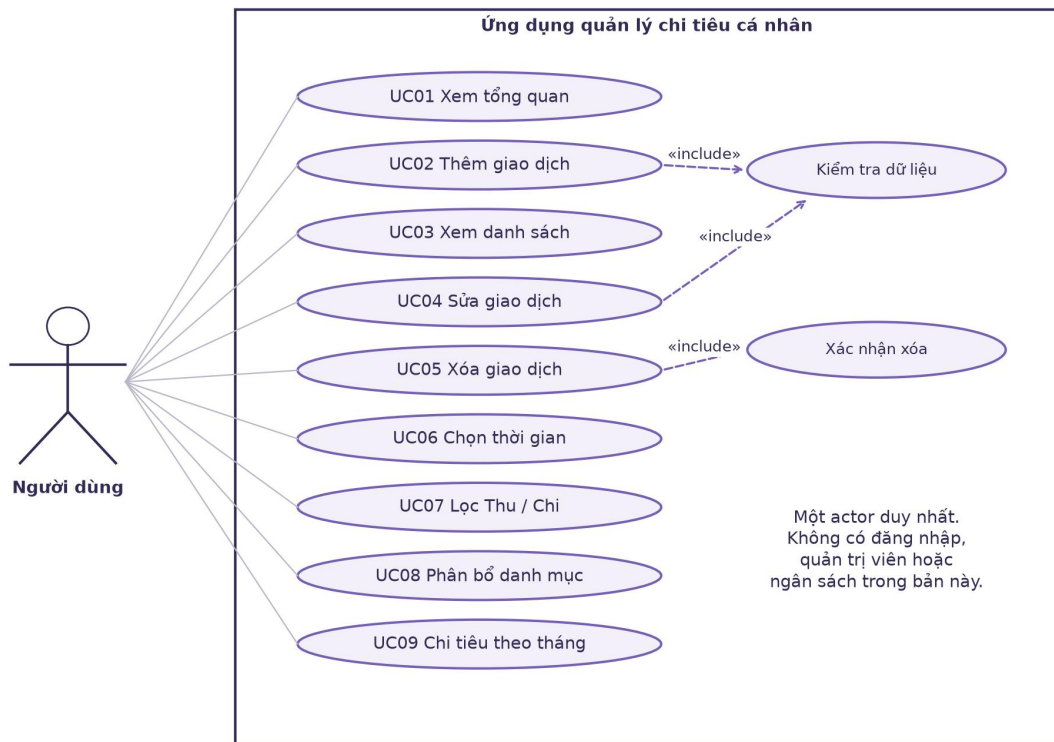
Tính toàn vẹn: không lưu số tiền âm/0 qua repository; danh mục được tham chiếu bằng khóa ngoại; cập nhật/xóa kiểm tra số dòng tác động. Khả năng sử dụng: giao diện tiếng Việt, phân biệt thu/chi bằng nhãn và màu, có thông báo ngay dưới ô tiền và xác nhận trước khi xóa. Khả năng bảo trì: chia file theo UI và data, tách summarize và AmountValidator để test mà không cần chạy UI.

Khả năng hoạt động cục bộ: các chức năng hiện có không cần mạng; manifest không khai báo quyền INTERNET. Hiệu năng: LazyColumn chỉ dựng các dòng cần hiển thị nhưng ViewModel vẫn nhận toàn bộ dữ liệu; vì vậy không khẳng định hệ thống đã tối ưu cho hàng trăm nghìn giao dịch. Bảo mật: dữ liệu nằm trong vùng riêng của ứng dụng, nhưng chưa mã hóa database và chưa khóa ứng dụng bằng sinh trắc học.

3.4. Actor và sơ đồ use case

Actor “Người dùng” khởi tạo các thao tác trực tiếp trên giao diện. Use case thêm và sửa dùng chung bước kiểm tra dữ liệu; use case xóa bao gồm bước xác nhận. Các bước kỹ thuật này được biểu diễn bằng quan hệ include để làm rõ hành vi luôn cần thực hiện, không phải chức năng ngân sách hoặc tài khoản.

SƠ ĐỒ USE CASE — QLCT



Hình 1. Use case tổng quát của bản Compose/MVVM/Room

3.5. Đặc tả UC01 — Xem tổng quan

Bảng 9. Thông tin use case UC01

Thuộc tính	Đặc tả
Mã / tên	UC01 — Xem tổng quan
Actor	Người dùng
Kích hoạt	Mở ứng dụng hoặc chọn tab Tổng quan
Tiền điều kiện	Database tải thành công; có thể có hoặc không có giao dịch
Hậu điều kiện	Không ghi database. Số liệu phản ánh period; danh sách gần đây còn chịu bộ lọc type đã chọn trước đó.

Luồng chính

1. Hệ thống nhận danh mục và giao dịch từ Flow của Room.
2. ViewModel đọc tháng, chế độ thời gian và loại lọc đang lưu.
3. summarize tính tổng thu, tổng chi, chênh lệch và dữ liệu biểu đồ.
4. Màn hình hiển thị thẻ tổng quan, donut, cột 6 tháng và tối đa 4 giao dịch gần đây.

5. Người dùng nhấn Xem tất cả để chuyển sang tab Giao dịch nếu cần xem toàn bộ danh sách.

Luồng thay thế và ngoại lệ

- Đang tải: hiển thị CircularProgressIndicator.
- Tải lỗi: hiển thị thông báo và nút Thử lại; không giả định database rỗng.
- Không có dữ liệu trong period: các tổng bằng 0, danh sách có lời nhắc thêm giao dịch và donut hiển thị vòng nền.

Điểm kiểm tra khi nghiệm thu

Đối chiếu tổng trên giao diện với dữ liệu mẫu; thử tháng rỗng và phạm vi toàn bộ.

3.6. Đặc tả UC02 — Thêm giao dịch

Bảng 10. Thông tin use case UC02

Thuộc tính	Đặc tả
Mã / tên	UC02 — Thêm giao dịch
Actor	Người dùng
Kích hoạt	Nhấn nút + trên thanh điều hướng dưới
Tiền điều kiện	Đã tải danh mục; không có thao tác ghi đang chạy
Hậu điều kiện	Một giao dịch mới được lưu bền vững nếu thao tác thành công; không có bản ghi mới khi validate thất bại hoặc người dùng hủy.

Luồng chính

1. Biểu mẫu mặc định loại EXPENSE, số tiền trống, danh mục đầu tiên, ngày hiện tại, ghi chú trống.
2. Người dùng chọn Chi tiền hoặc Thu tiền.
3. Người dùng nhập số tiền nguyên VND; chọn danh mục từ DropdownMenu.
4. Người dùng có thể chọn lại ngày bằng DatePickerDialog và nhập ghi chú.
5. Nhấn Lưu: hệ thống đánh dấu đã kiểm tra tiền, gọi AmountValidator và xác nhận danh mục trong danh sách.
6. Nếu hợp lệ, tạo TransactionEntity với id=0 và chuyển tới ViewModel.save().
7. ViewModel bật busy; repository kiểm tra tiền, loại, ngày, danh mục và trim ghi chú.
8. DAO.insert() sinh ID; thao tác hoàn tất làm completedActions tăng.

9. Dialog đóng; Flow phát danh sách mới; các tổng và biểu đồ tự tính lại.

Luồng thay thế và ngoại lệ

- Số tiền rỗng: “Vui lòng nhập số tiền”.
- Số âm hoặc 0: “Số tiền phải lớn hơn 0”.
- Chuỗi không chuyển được sang Long: “Nhập số tiền nguyên hợp lệ (VND)”.
- Tiền vượt 999.999.999.999: báo mức tối đa.
- Thiếu danh mục: yêu cầu chọn; lỗi ghi: `actionError` và giữ dữ liệu nhập để thử lại.
- Nhân Hủy hoặc đóng dialog khi không busy: không ghi bản ghi mới.

Điểm kiểm tra khi nghiệm thu

Kiểm tra khoản chi/thu hợp lệ; nhập 0, âm, rỗng, chữ, số thập phân và vượt giới hạn; kiểm tra tránh nhấn Lưu lặp.

3.7. Đặc tả UC03 — Xem lịch sử giao dịch

Bảng 11. Thông tin use case UC03

Thuộc tính	Đặc tả
Mã / tên	UC03 — Xem lịch sử giao dịch
Actor	Người dùng
Kích hoạt	Chọn tab Giao dịch hoặc Xem tất cả
Tiền điều kiện	Đã tải dữ liệu
Hậu điều kiện	Không ghi database nếu người dùng chỉ xem hoặc cuộn.

Luồng chính

1. Màn hình giữ thẻ tổng quan và vùng chọn thời gian.
2. Hiện thị các chip Tất cả, Thu, Chi và số giao dịch phù hợp.
3. Mỗi dòng có icon danh mục, tên, ngày dd/MM, ghi chú rút gọn, số tiền có dấu và nút thùng rác.
4. Danh sách theo t.date DESC rồi t.id DESC từ truy vấn DAO.
5. Người dùng cuộn LazyColumn để xem các giao dịch còn lại.
6. Chạm dòng mở sửa; chạm thùng rác mở xác nhận xóa.

Luồng thay thế và ngoại lệ

- Danh sách rỗng sau lọc: hiển thị thẻ Chưa có giao dịch và gợi ý nút +.
- Ghi chú dài bị rút gọn trên dòng nhưng được nạp đầy đủ khi mở biểu mẫu sửa.
- Trong lúc busy, các hành động sửa/xóa bị vô hiệu hóa.

Điểm kiểm tra khi nghiệm thu

Kiểm tra thứ tự cùng ngày, số lượng, dấu Thu/Chi và nội dung ghi chú khi mở sửa.

3.8. Đặc tả UC04 — Sửa giao dịch

Bảng 12. Thông tin use case UC04

Thuộc tính	Đặc tả
Mã / tên	UC04 — Sửa giao dịch
Actor	Người dùng
Kích hoạt	Chạm một dòng giao dịch
Tiền điều kiện	Giao dịch tồn tại trong danh sách visible; không busy
Hậu điều kiện	Chỉ bản ghi đúng ID bị thay đổi. Khoản thu/chi và tổng hợp được tính theo giá trị mới.

Luồng chính

1. ExpenseScreen ghi editingId và mở TransactionEditor với existing tương ứng.
2. Biểu mẫu nạp loại, tiền, categoryId, date và note của bản ghi cũ.
3. Người dùng thay đổi một hoặc nhiều trường và nhấn Lưu.
4. Áp dụng cùng quy tắc tiền và danh mục như thêm mới.
5. TransactionEntity giữ existing.id; repository nhận biết id khác 0.
6. DAO.update() trả số dòng; repository yêu cầu đúng 1 dòng.
7. Thành công: tăng completedActions, đóng biểu mẫu và nhận dữ liệu Flow mới.

Luồng thay thế và ngoại lệ

- Nhập dữ liệu không hợp lệ: giữ nội dung chỉnh sửa, hiện lỗi và không update.
- Bản ghi đã không còn tồn tại: update trả 0, check thất bại và ViewModel hiển thị actionError.
- Đổi loại hoặc ngày có thể làm giao dịch không còn phù hợp bộ lọc hiện tại; biến mất khỏi danh sách không đồng nghĩa bị xóa.

- Hủy: không ghi các chỉnh sửa chưa lưu.

Điểm kiểm tra khi nghiệm thu

Kiểm tra sửa tiền, loại, danh mục, ngày và note; thử tiền âm/0 khi update; kiểm tra ID không đổi.

3.9. Đặc tả UC05 — Xóa giao dịch

Bảng 13. Thông tin use case UC05

Thuộc tính	Đặc tả
Mã / tên	UC05 — Xóa giao dịch
Actor	Người dùng
Kích hoạt	Nhấn biểu tượng thùng rác của một dòng
Tiền điều kiện	Giao dịch tồn tại; không busy
Hậu điều kiện	Bản ghi bị xóa bền vững sau khi thành công. Không có chức năng Undo trong bản hiện tại.

Luồng chính

1. Hệ thống ghi deletingId và hiển thị dialog Xóa giao dịch?.
2. Dialog hiển thị tên danh mục, số tiền và thông báo không thể hoàn tác.
3. Người dùng nhấn Xóa để xác nhận.
4. ViewModel bật busy và gọi repository.delete(id).
5. DAO.delete(id) thực thi truy vấn có tham số; repository kiểm tra kết quả đúng 1 dòng.
6. Thành công: đóng dialog, Room phát dữ liệu mới, tổng quan và biểu đồ loại bỏ khoản giao dịch.

Luồng thay thế và ngoại lệ

- Hủy hoặc đóng dialog khi không busy: deletingId được xóa khỏi state UI, database không đổi.
- Không còn bản ghi để xóa hoặc ghi thất bại: hiện actionError, bật lại nút để thử lại.
- Trong khi busy, không cho xác nhận lặp hay đóng bằng thao tác hủy trong dialog.

Điểm kiểm tra khi nghiệm thu

Kiểm tra cả xác nhận và hủy; xóa khoản thu/chi; xóa giao dịch cuối cùng; kiểm tra không seed lại sau khi danh sách rỗng.

3.10. Đặc tả UC06 — Chọn khoảng thời gian

Bảng 14. Thông tin use case UC06

Thuộc tính	Đặc tả
Mã / tên	UC06 — Chọn khoảng thời gian
Actor	Người dùng
Kích hoạt	Nhấn < / > hoặc nút Theo tháng / Toàn bộ
Tiền điều kiện	Dữ liệu đang tải thành công
Hậu điều kiện	Không sửa bất kỳ giao dịch nào. Chỉ thay đổi phạm vi hiển thị và tính toán.

Luồng chính

1. Tháng mặc định khi chưa có state là YearMonth.now().
2. Nút < gọi changeMonth(-1); nút > gọi changeMonth(1).
3. Nút Theo tháng/Toàn bộ đảo monthOnly bằng setMonthOnly().
4. SavedStateHandle lưu các giá trị; combine nhận tham số mới.
5. Nếu monthOnly=true, period chỉ có date bắt đầu bằng yyyy-MM- của tháng chọn.
6. Nếu monthOnly=false, period là toàn bộ rows; tháng chọn vẫn làm mốc kết thúc của biểu đồ cột.

Luồng thay thế và ngoại lệ

- Tháng không có dữ liệu: tổng thu/chi bằng 0, danh sách rỗng nhưng biểu đồ cột vẫn có 6 tháng.
- Đổi tháng qua tháng 12/tháng 1 được YearMonth xử lý cả năm.
- Không giới hạn chọn tháng tương lai; có thể ghi hoặc xem các ngày tương lai trong bản hiện tại.

Điểm kiểm tra khi nghiệm thu

Kiểm tra ranh giới năm, tháng 2 năm nhuận, tháng rỗng và chế độ toàn bộ.

3.11. Đặc tả UC07 — Lọc loại giao dịch

Bảng 15. Thông tin use case UC07

Thuộc tính	Đặc tả
Mã / tên	UC07 — Lọc loại giao dịch
Actor	Người dùng
Kích hoạt	Nhấn chip Tất cả / Thu / Chi ở tab Giao dịch
Tiền điều kiện	Đã tải dữ liệu và có period đang chọn
Hậu điều kiện	Database không đổi. Chỉ danh sách và số lượng visible đổi; income, expense và chart không bị thay đổi bởi type.

Luồng chính

1. Tất cả đặt type=ALL; Thu đặt INCOME; Chi đặt EXPENSE.
2. SavedStateHandle phát type mới cho combine.
3. summarize lọc period thành visible theo type.
4. LazyColumn hiển thị visible, số giao dịch thay đổi tương ứng.
5. Thẻ tổng quan, donut và cột vẫn dùng dữ liệu trước bước lọc type.

Luồng thay thế và ngoại lệ

- Không có loại giao dịch tương ứng: hiện trạng thái rỗng, không xóa dữ liệu.
- Bộ lọc còn tác động danh sách gần đây ở Tổng quan khi quay về tab này.
- Người dùng không có bộ lọc danh mục hoặc ô tìm kiếm trong bản hiện tại.

Điểm kiểm tra khi nghiệm thu

So sánh tổng quan trước/sau chọn Thu; kiểm tra cả ALL, INCOME, EXPENSE trong tháng và toàn bộ.

3.12. Đặc tả UC08 — Xem phân bổ chi theo danh mục

Bảng 16. Thông tin use case UC08

Thuộc tính	Đặc tả
Mã / tên	UC08 — Xem phân bổ chi theo danh mục
Actor	Người dùng
Kích hoạt	Xem donut ở Tổng quan hoặc Thống kê
Tiền điều kiện	Dữ liệu đã được tổng hợp

Thuộc tính	Đặc tả
Hậu điều kiện	Không ghi dữ liệu; tổng số tiền các nhóm bằng tổng expense trong period.

Luồng chính

1. Lấy các giao dịch EXPENSE trong period.
2. Nhóm theo categoryId bằng groupBy, cộng amount trong mỗi nhóm.
3. Tạo CategoryExpense(id,name,amount), sắp giảm dần theo amount.
4. Vẽ mỗi cung bằng tỷ lệ số tiền danh mục trên tổng expense; màu dựa theo categoryId.
5. Chú giải hiển thị tên và tỷ lệ phần trăm; tab Thống kê hiển thị thêm số tiền chính xác từng nhóm.

Luồng thay thế và ngoại lệ

- expense=0: vẽ vòng nền, nhắc thêm khoản chi và tránh chia cho 0.
- Tỷ lệ chú giải dùng toInt nên làm tròn xuống; tổng phần trăm hiển thị có thể chưa bằng 100.
- Tên dài được rút gọn trong chú giải; categoryId vẫn xác định nhóm dữ liệu.

Điểm kiểm tra khi nghiệm thu

Kiểm tra nhiều giao dịch cùng danh mục, loại thu không tham gia, tháng không có chi và chọn toàn bộ.

3.13. Đặc tả UC09 — Xem chi tiêu theo tháng

Bảng 17. Thông tin use case UC09

Thuộc tính	Đặc tả
Mã / tên	UC09 — Xem chi tiêu theo tháng
Actor	Người dùng
Kích hoạt	Xem biểu đồ cột ở Tổng quan hoặc Thống kê
Tiền điều kiện	Có tháng được chọn làm mốc
Hậu điều kiện	Database không đổi; cột chi thể hiện khoản chi, không thể hiện thu hoặc số dư.

Luồng chính

1. Tạo 6 YearMonth từ month.minusMonths(5) đến month.

2. Với mỗi tháng, lấy các rows có type=EXPENSE và date bắt đầu bằng prefix tháng đó.
3. Cộng amount tạo MonthlyExpense; không có dữ liệu thì nhận 0.
4. Tính giá trị lớn nhất để chuẩn hóa chiều cao cột; nếu tất cả 0 thì max được giữ tối thiểu 1 để tránh chia cho 0.
5. Vẽ cột, nhãn tháng/năm và mô tả truy cập; Thống kê hiển thị thêm giá trị VND từng tháng.

Luồng thay thế và ngoại lệ

- Tất cả tháng có chi=0: hiện thông báo Chưa có khoản chi trong giai đoạn này.
- Bộ lọc type=INCOME không làm các cột về 0.
- Chọn Toàn bộ không đổi số tháng của biểu đồ; nó vẫn là cửa sổ 6 tháng đến month.

Điểm kiểm tra khi nghiệm thu

Kiểm tra 6 điểm, ranh giới năm, tháng không có giao dịch và giá trị tiền lớn.

3.14. Ma trận truy vết yêu cầu

Bảng 18. Truy vết yêu cầu – use case – mã nguồn – kiểm tra

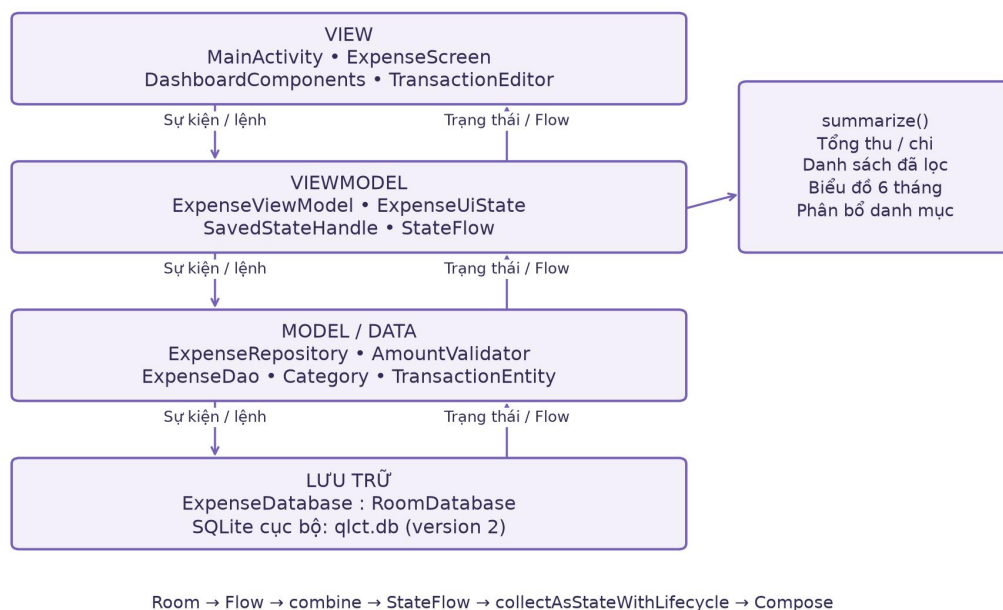
Yêu cầu	Use case	Mã chính	Cách kiểm tra
FR01	UC01	ExpenseSummary / DashboardHeader	Unit test + ảnh tổng quan
FR02, FR04	UC02, UC04	TransactionEditor / Repository / DAO	Test Room + thao tác UI thêm
FR03	UC03	observeTransactions / TransactionTile	Ảnh danh sách và thứ tự truy vấn
FR05	UC05	delete / AlertDialog	Test Room + xóa giao dịch thử trên UI
FR06, FR07	UC06, UC07	SavedStateHandle / summarize	Unit test period và type
FR08	UC08	categoryExpenses / CategoryChart	Unit test groupBy và period
FR09	UC09	chart / SpendingChart	Unit test 6 tháng và ranh giới năm
FR10	UC02, UC04, UC05	AmountValidator / actionError / loadError	Unit test và ảnh lỗi 0/âm

CHƯƠNG 4. THIẾT KẾ KIẾN TRÚC VÀ CƠ SỞ DỮ LIỆU

4.1. Kiến trúc tổng thể

Ứng dụng triển khai theo kiến trúc MVVM ở quy mô một module app. MainActivity là điểm vào, kế thừa ComponentActivity và gọi setContent. Activity tạo ExpenseRepository từ DAO của singleton ExpenseDatabase, rồi cung cấp factory để Compose lấy ExpenseViewModel. Các thành phần này tạo một luồng phụ thuộc từ giao diện xuống dữ liệu, trong khi dữ liệu quan sát đi theo hướng ngược lại.

KIẾN TRÚC MVVM VÀ DÒNG DỮ LIỆU



Hình 2. Kiến trúc MVVM và hai chiều sự kiện – trạng thái

Đường ghi dữ liệu: người dùng → TransactionEditor → ExpenseViewModel → ExpenseRepository → ExpenseDao → SQLite. Đường cập nhật màn hình: Room nhận thay đổi → Flow danh mục/giao dịch → combine với bộ lọc → summarize → StateFlow<ExpenseUiState> → collectAsStateWithLifecycle → Compose. Việc tách đường lệnh và trạng thái giúp tránh cập nhật số tổng trực tiếp ở từng nút.

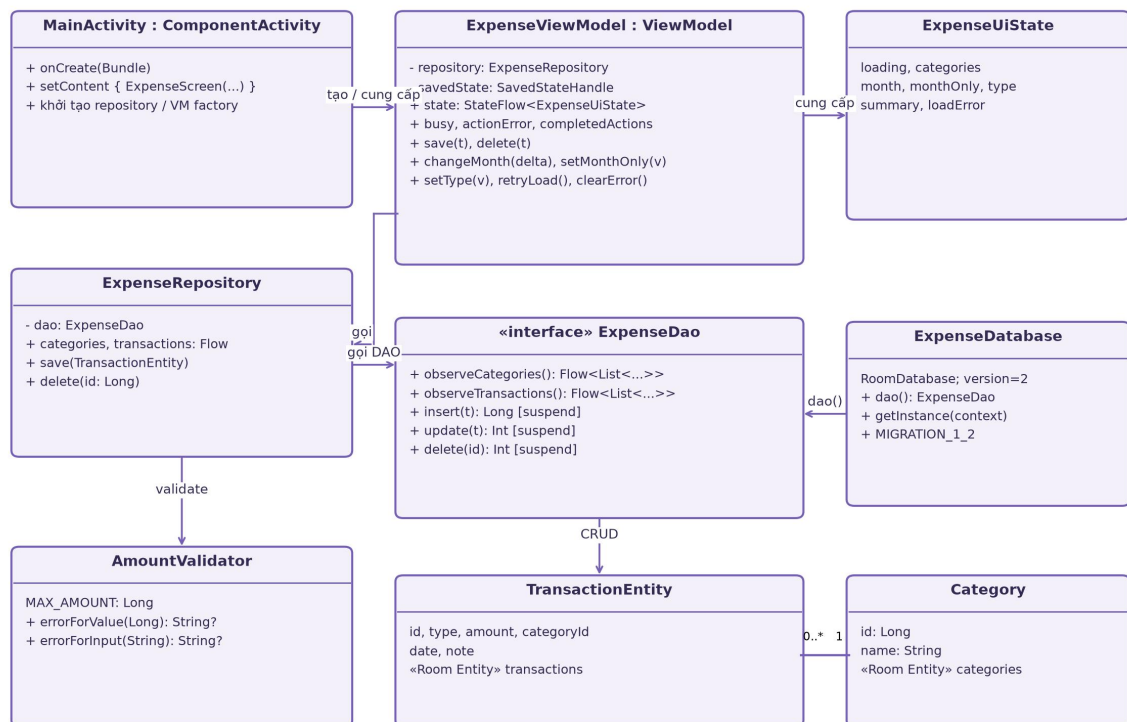
4.2. Trách nhiệm của từng thành phần

Bảng 19. Trách nhiệm lớp và thành phần giao diện

Tên	Trách nhiệm	Không chịu trách nhiệm
MainActivity	Khởi tạo dependency, setContent, theme, edge-to-edge	Không validate tiền; không viết truy vấn SQL.

Tên	Trách nhiệm	Không chịu trách nhiệm
ExpenseScreen	Tab, danh sách, loading/error, editor/delete dialog	Không trực tiếp insert/update/delete database.
DashboardComponents	Header, thẻ, donut, cột, dòng giao dịch, icon, navigation	Không quản lý phiên bản database.
TransactionEditor	State nhập, chọn loại/ngày/danh mục, validate ở UI	Không tự sinh ID cho bản ghi mới.
ExpenseViewModel1	StateFlow, bộ lọc, busy, gọi repository, lỗi, hoàn tất	Không vẽ biểu đồ bằng Canvas.
ExpenseRepository	Validate và chuẩn hóa; quyết định insert/update; kiểm tra số dòng	Không điều khiển đóng dialog.
AmountValidator	Quy tắc tiền đầu vào và giá trị Long	Không gọi Room hoặc biết tab nào đang mở.
ExpenseDao	Quan sát danh sách/danh mục; insert/update/delete	Không quyết định màu, hình biểu đồ hoặc nhãn tiếng Việt.
ExpenseDatabase	Singleton, schema, callback seed, migration	Không giữ số tiền đang nhập trên biểu mẫu.

SƠ ĐỒ LỚP — CÁC LỚP CHÍNH THEO MÃ NGUỒN



Hình 3. Sơ đồ lớp chính và quan hệ phụ thuộc trong mã hiện tại

TransactionEntity là đối tượng ánh xạ bảng; TransactionRow là kết quả truy vấn JOIN có thêm categoryName để giao diện không phải tự ghép tên. CategoryExpense và MonthlyExpense là mô hình dữ liệu cho thống kê, không phải các bảng lưu trữ.

ExpenseSummary chứa kết quả tổng hợp; ExpenseUiState gói dữ liệu cùng trạng thái loading và lỗi tải.

4.3. Thiết kế trạng thái giao diện

Bảng 20. Các trường ExpenseUiState

Trường	Kiểu / mặc định	Ý nghĩa
loading	Boolean / true	Đang đợi dữ liệu ban đầu.
categories	List<Category> / emptyList	Danh mục để hiển thị và chọn khi thêm/sửa.
month	YearMonth / now()	Mốc tháng của phạm vi và cuối biểu đồ cột.
monthOnly	Boolean / true	true: chỉ period tháng; false: toàn bộ giao dịch.
type	String / ALL	ALL, INCOME hoặc EXPENSE cho danh sách visible.
summary	ExpenseSummary / 0 và các list rỗng	income, expense, visible, chart và categoryExpenses.
loadError	String? / null	Thông báo lỗi đọc dữ liệu khi Flow phát lỗi.

Bảng 21. Các trạng thái thao tác và vị trí lưu

Trạng thái	Nơi lưu	Hành vi
month, monthOnly, type	SavedStateHandle trong ViewModel	Cập nhật bằng changeMonth / setMonthOnly / setType.
busy	MutableStateFlow trong ViewModel	Chặn thao tác ghi tiếp theo; disable nút khi đang ghi.
actionError	MutableStateFlow<String?>	Hiện thông báo lỗi ghi; giữ dữ liệu nhập.
completedActions	MutableStateFlow<Long>	Tăng sau save/delete thành công; UI đóng dialog.
tab, editorOpen, editingId, deletingId	rememberSaveable trong ExpenseScreen	Giữ lựa chọn trang và hộp thoại qua tái tạo state được hỗ trợ.
type, amount, categoryId, date, note, amountChecked	rememberSaveable trong TransactionEditor	Giữ nội dung nhập và trạng thái validate.

Trạng thái lỗi tải và lỗi ghi được tách biệt. Khi đọc lỗi, state xuất ra loadError và có nút Thử lại; retryLoad tăng một giá trị MutableStateFlow để flatMapLatest đăng ký lại các luồng. Khi ghi lỗi, ViewModel giữ actionError và trả busy=false trong finally, cho phép người dùng thử lại. CancellationException được ném lại thay vì xử lý như lỗi nghiệp vụ thông thường.

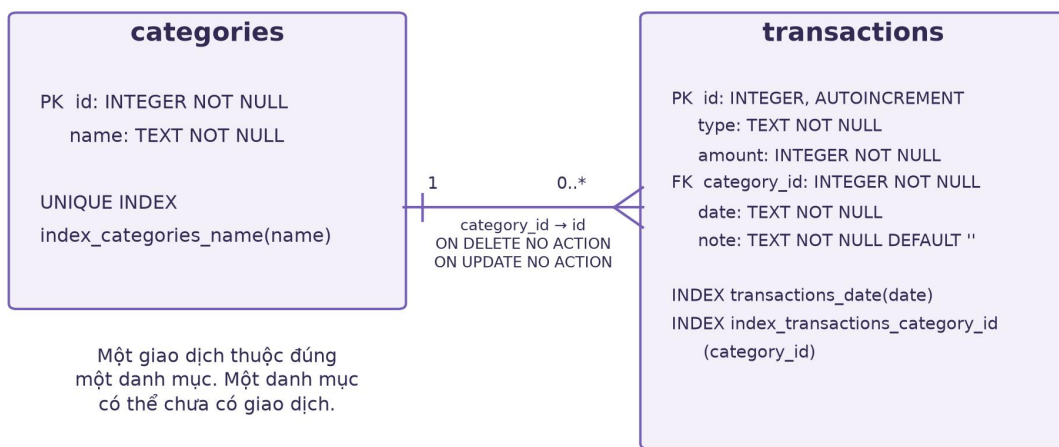
Các flow busy, actionError và completedActions hiện được công khai dưới dạng MutableStateFlow; có thể cải tiến bằng cách chỉ công khai StateFlow và giữ nguồn

mutable private. Đây là cải tiến về đóng gói, không phải chức năng đã có thêm trong phiên bản báo cáo.

4.4. Mô hình quan hệ dữ liệu

Database có đúng hai bảng nghiệp vụ: categories và transactions. Một danh mục có từ 0 đến nhiều giao dịch; mỗi giao dịch bắt buộc tham chiếu một danh mục tồn tại. ID của danh mục được seed cố định, còn ID giao dịch được SQLite sinh tự động khi thêm mới.

ERD — CƠ SỞ DỮ LIỆU ROOM VERSION 2



amount > 0 và type ∈ {INCOME, EXPENSE}: kiểm tra ở ứng dụng; không có SQL CHECK trong schema v2.

Hình 4. ERD theo schema Room version 2

4.4.1. Từ điển bảng categories

Bảng 22. Từ điển dữ liệu categories

Cột	Kotlin / SQLite	Ràng buộc	Ý nghĩa / ví dụ
id	Long / INTEGER	PRIMARY KEY, NOT NULL; không autoGenerate	Mã danh mục; ví dụ 1 = Ăn uống.
name	String / TEXT	NOT NULL; UNIQUE qua index_categories_name	Tên hiển thị; ví dụ Di chuyển.

Bảng categories không có cột type. Vì vậy trong ứng dụng hiện tại danh mục không bị khóa riêng cho Thu hoặc Chi; người dùng có thể chọn bất kỳ danh mục nào trong bảy danh mục đã nạp. Không mô tả tính năng tự lọc danh mục theo loại vì mã chưa triển khai hành vi đó.

4.4.2. Từ điển bảng transactions

Bảng 23. Từ điển dữ liệu transactions

Cột	Kotlin / SQLite	Ràng buộc	Ý nghĩa / ví dụ
id	Long / INTEGER	PK, AUTOINCREMENT, NOT NULL	0 ở entity trước insert; database sinh ID dương.
type	String / TEXT	NOT NULL; kiểm tra giá trị tại repository	INCOME hoặc EXPENSE.
amount	Long / INTEGER	NOT NULL; kiểm tra 1..99999999999 trong ứng dụng	45000 VND; không lưu - 45000.
category_id	Long / INTEGER	NOT NULL; FK → categories.id	1 liên kết danh mục Ăn uống.
date	String / TEXT	NOT NULL; LocalDate.parse tại repository	yyyy-MM-dd, ví dụ 2026-10-06.
note	String / TEXT	NOT NULL DEFAULT ""; trim khi lưu	“Tiền ăn trưa” hoặc chuỗi rỗng.

4.4.3. Khóa và chỉ mục

Bảng 24. Các chỉ mục và hành động khóa ngoại

Tên / quan hệ	Định nghĩa	Mục đích / hệ quả
index_categories_name	UNIQUE(name)	Tên danh mục không trùng ở mức database.
transactions_date	INDEX(date)	Hỗ trợ truy vấn/sắp xếp ngày; bộ lọc tháng hiện tại vẫn tính trong bộ nhớ.
index_transactions_category_id	INDEX(category_id)	Hỗ trợ JOIN và kiểm tra tham chiếu danh mục.
FK transactions.category_id	REFERENCES categories(id)	Bản ghi phải tham chiếu danh mục tồn tại.
ON DELETE / UPDATE	NO ACTION / NO ACTION	Không cascade giao dịch khi xóa/đổi ID danh mục; hiện UI không cung cấp CRUD danh mục.

4.5. Phân biệt ràng buộc database và validate ứng dụng

Schema Room v2 có PK, FK, UNIQUE, NOT NULL và DEFAULT của note. Nó không chứa CHECK(amount > 0) hoặc CHECK(type IN (...)) như bảng SQLiteOpenHelper v1. Vì vậy phải mô tả đúng: số tiền dương và loại hợp lệ được bảo vệ bằng AmountValidator/ExpenseRepository; việc gọi thẳng DAO bỏ qua repository có thể vượt các quy tắc nghiệp vụ này.

Trong luồng UI bình thường, bước kiểm tra diễn ra hai lần với mục đích khác nhau. UI kiểm tra chuỗi để báo lỗi dễ hiểu ngay tại trường nhập; repository kiểm tra giá trị đã

parse để mọi thao tác lưu của ứng dụng áp dụng cùng quy tắc. Khóa ngoại là lớp bảo vệ bổ sung ở database nếu categoryId không tồn tại. Ngày là TEXT nên SQLite không tự xác nhận yyyy-MM-dd; kiểm tra ngày cũng ở repository.

Để mở rộng khả năng chống dữ liệu không hợp lệ qua các đường ghi mới, nhóm có thể cân nhắc thiết kế trigger hoặc ràng buộc ở database kèm migration. Báo cáo không coi phương án mở rộng này đã được triển khai. Các API ghi hiện tại phải đi qua repository thay vì để UI truy cập DAO trực tiếp.

4.6. DDL và truy vấn theo schema xuất từ KSP

4.6.1. Tạo bảng categories

```
CREATE TABLE IF NOT EXISTS `categories` (
  `id` INTEGER NOT NULL,
  `name` TEXT NOT NULL, PRIMARY KEY(`id`));

CREATE UNIQUE INDEX IF NOT EXISTS `index_categories_name` ON `categories` (`name`);
```

4.6.2. Tạo bảng transactions

```
CREATE TABLE IF NOT EXISTS `transactions` (
  `id` INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
  `type` TEXT NOT NULL,
  `amount` INTEGER NOT NULL,
  `category_id` INTEGER NOT NULL,
  `date` TEXT NOT NULL,
  `note` TEXT NOT NULL DEFAULT '',
  FOREIGN KEY(`category_id`) REFERENCES `categories`(`id`) ON UPDATE NO ACTION ON DELETE NO ACTION );

CREATE INDEX IF NOT EXISTS `transactions_date` ON `transactions` (`date`);

CREATE INDEX IF NOT EXISTS `index_transactions_category_id` ON `transactions`
(`category_id`);
```

4.6.3. Truy vấn đọc danh sách

```
SELECT t.*, c.name AS categoryName
FROM transactions t
JOIN categories c ON c.id = t.category_id
ORDER BY t.date DESC, t.id DESC;
```

DAO ánh xạ kết quả sang TransactionRow. JOIN bảo đảm giao diện nhận được categoryName cùng transaction. Quy tắc thứ tự ưu tiên ngày, sau đó ID, nghĩa là giao dịch vừa thêm cùng ngày thường đứng trước bản ghi cùng ngày có ID nhỏ hơn. Đây không phải thứ tự ghi chú hay tên danh mục.

4.6.4. Ghi dữ liệu và truy vấn tham số

```
@Insert suspend fun insert(transaction: TransactionEntity): Long
@Update suspend fun update(transaction: TransactionEntity): Int
@Query("DELETE FROM transactions WHERE id = :id")
suspend fun delete(id: Long): Int
```

Room sinh truy vấn insert/update từ annotation; phương thức delete dùng tham số :id thay vì nối chuỗi SQL từ người dùng. Ghi chú có dấu nháy không trở thành câu lệnh SQL vì là dữ liệu được bind. Repository yêu cầu update/delete tác động đúng một dòng để tránh báo thành công khi ID không còn tồn tại.

4.7. Dữ liệu khởi tạo và ví dụ tính toán

Bảng 25. Bẫy danh mục có sẵn

ID	Tên danh mục
1	Ăn uống
2	Di chuyển
3	Học tập
4	Mua sắm
5	Sinh hoạt
6	Lương / Trợ cấp
7	Khác

Bảng 26. Tám giao dịch mẫu khi tạo database lần đầu

Loại	Số tiền (VND)	Danh mục	Ghi chú
INCOME	3.000.000	Lương / Trợ cấp	Dữ liệu mẫu: trợ cấp tháng
INCOME	1.200.000	Lương / Trợ cấp	Dữ liệu mẫu: làm thêm
EXPENSE	45.000	Ăn uống	Dữ liệu mẫu: ăn trưa
EXPENSE	70.000	Di chuyển	Dữ liệu mẫu: đổ xăng
EXPENSE	150.000	Học tập	Dữ liệu mẫu: sách
EXPENSE	250.000	Mua sắm	Dữ liệu mẫu: đồ dùng
EXPENSE	800.000	Sinh hoạt	Dữ liệu mẫu: tiền phòng
EXPENSE	30.000	Ăn uống	Dữ liệu mẫu: ăn sáng

Các ngày mẫu được gán theo ngày và đầu tháng hiện tại của thiết bị tại lần tạo database. Vì thế giá trị tổng mẫu thuộc tháng khởi tạo, không phải tháng cố định trong mã. Tổng thu mẫu = 4.200.000 VND; tổng chi mẫu = 1.345.000 VND; chênh lệch = 2.855.000 VND. Sau khi người dùng thêm/sửa/xóa, tổng thực tế có thể khác các giá trị mẫu này.

onCreate đọc assets/seed.sql, tách câu lệnh theo dấu chấm phẩy và chạy các INSERT cố định của ứng dụng. Đây không phải cơ chế chạy file SQL do người dùng cung cấp. Callback chỉ chạy khi tạo database mới; mở lại hoặc xóa hết giao dịch không tự nạp lại tám bản ghi mẫu.

4.7.1. Diễn giải INSERT danh mục trong seed.sql

INSERT INTO categories VALUES tạo bảy danh mục ID 1–7: Ăn uống, Di chuyển, Học tập, Mua sắm, Sinh hoạt, Lương / Trợ cấp và Khác. Danh mục tạo trước giao dịch vì category_id có FK tới categories.id. Mã 6 dùng cho hai khoản thu mẫu; mã 1–5 dùng cho chi.

Giao diện cho chọn tên nhưng database lưu ID; DAO JOIN lấy categoryName khi đọc. Không chép tên vào mỗi giao dịch. Phiên bản hiện tại chưa có CRUD danh mục trên giao diện.

4.7.2. Diễn giải INSERT giao dịch và tổng mẫu

INSERT INTO transactions(type,amount,category_id,date,note) cung cấp năm cột; bỏ id để SQLite tự sinh. type quy định thu/chi; amount là VND nguyên dương; category_id nối danh mục; date ghi ngày; note diễn giải khoản tiền.

Trợ cấp là INCOME 3000000, danh mục 6, ngày đầu tháng; làm thêm là INCOME 1200000, danh mục 6, ngày hiện tại. Tổng thu 4.200.000. Sáu EXPENSE mô phỏng ăn trưa, xăng, sách, đồ dùng, phòng và ăn sáng, tổng chi 1.345.000. Chênh lệch là 2.855.000 VND.

Ăn trưa và ăn sáng cùng ID danh mục 1 nên nhóm Ăn uống là 75.000. Di chuyển 70.000; Học tập 150.000; Mua sắm 250.000; Sinh hoạt 800.000. Tổng nhóm bằng tổng chi; hai khoản thu không vào donut. Ghi chú “Dữ liệu mẫu:” nhận diện dữ liệu giả lập, không phải khảo sát hay tài chính thật của thành viên.

4.7.3. Ngày khởi tạo và quan hệ với chart tháng

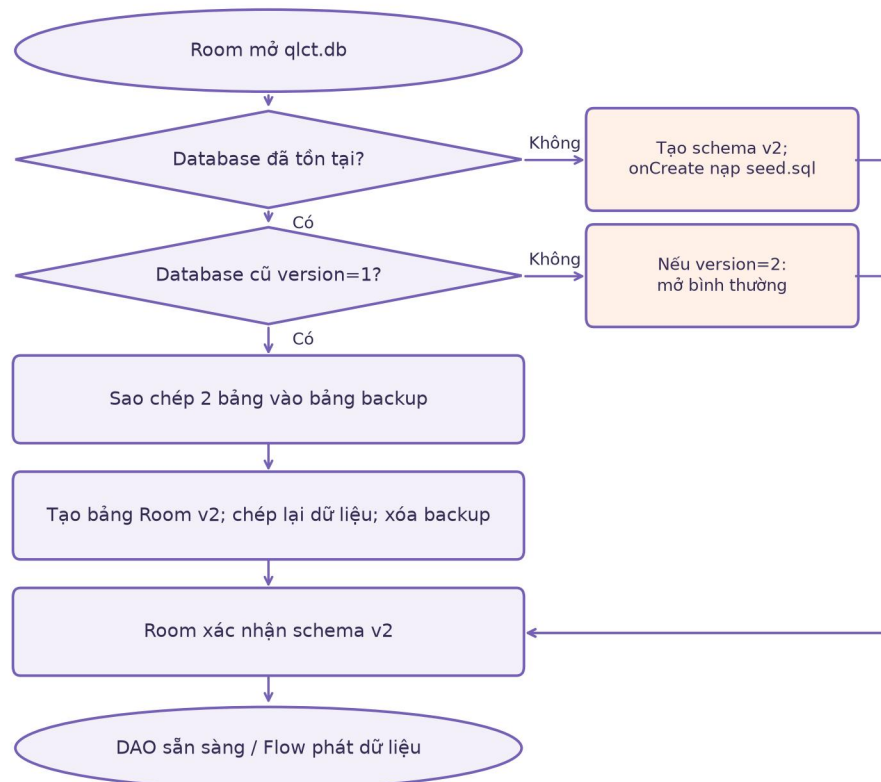
date('now','localtime') lấy ngày theo giờ địa phương khi tạo database; thêm 'start of month' lấy ngày đầu tháng. seed.sql không cố định 10/2026: tạo database tháng khác sẽ sinh tám dòng trong tháng mới.

Database mới có tám dòng thuộc tháng khởi tạo. Tổng thu/chi/chênh lệch là 4.200.000/1.345.000/2.855.000. Chart có cột tháng đó 1.345.000 và các tháng trước 0 nếu chưa có dữ liệu. Sau thao tác của người dùng, giao diện theo dữ liệu thực, không bắt buộc bằng số mẫu.

Room callback onCreate nạp seed một lần khi tạo database. Mở lại, chuyển tab, xóa hết giao dịch hay migration không nạp lại mẫu, tránh trùng và không ghi đè lịch sử. Giao dịch về sau đi qua View → ViewModel → Repository → DAO → Room.

4.8. Thiết kế migration version 1 sang version 2

FLOW — KHỞI TẠO VÀ MIGRATION DATABASE



Hình 5. Luồng mở database mới, đã có version 2 hoặc migration version 1

1. Room kiểm tra schema/database khi mở qlct.db. Với bản cũ user_version=1, chạy MIGRATION_1_2.
2. CREATE TABLE ..._backup AS SELECT * sao chép categories và transactions.
3. DROP transactions trước categories để giữ trật tự xử lý quan hệ tham chiếu.
4. Tạo lại categories và transactions theo nullability, default, FK và chỉ mục của Room v2.
5. INSERT INTO categories SELECT ... rồi INSERT INTO transactions SELECT ... để phục hồi các ID và dữ liệu.
6. Xóa hai bảng backup. Room kiểm tra schema mới và quản lý identity hash của version 2.

7. Mở DAO và phát Flow; không chạy seed.sql cho database đang nâng cấp.

Bảng backup ở đây là các bảng trung gian trong quá trình migration, không phải file sao lưu bên ngoài cho người dùng. Room quản lý migration trong cơ chế nâng cấp database của nó. Khi không tồn tại đường migration cần thiết, ứng dụng báo lỗi mở dữ liệu thay vì tự xóa database. Hiện chỉ có đường 1→2; mọi nâng version trong tương lai phải bổ sung migration và kiểm thử tương ứng.

Kiểm thử hiện có xác nhận dữ liệu và ID đang tồn tại được giữ nguyên, ghi chú có nội dung đặc biệt không mất, database rỗng không bị seed lại, schema sau migration đúng cột/chi mục/FK/default. Báo cáo không khẳng định bảo toàn mọi thông tin phụ trợ của SQLite hoặc đã thử mọi tình huống bị ngắt điện giữa migration.

4.9. Bảo vệ dữ liệu và các giới hạn lưu trữ

File database nằm trong sandbox ứng dụng; QLCT không có chức năng gửi dữ liệu ra máy chủ. Manifest hiện dùng allowBackup=false, nhưng lint cảnh báo về dataExtractionRules trên Android mới; do đó không diễn đạt rằng mọi cơ chế chuyển dữ liệu giữa thiết bị đều đã được cấu hình hoàn chỉnh. Dữ liệu chưa được mã hóa bằng SQLCipher và không có cơ chế xuất file sao lưu trong giao diện.

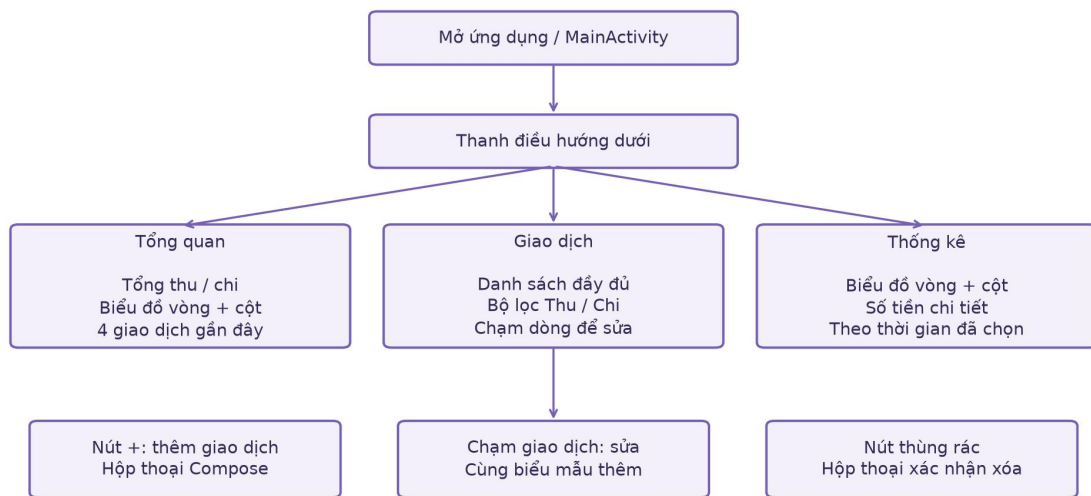
Cài APK cập nhật với cùng applicationId và chữ ký có thể giữ dữ liệu ứng dụng. Gỡ cài đặt hoặc dùng Clear storage sẽ xóa dữ liệu riêng; khi cài/chạy lại, database mới được tạo và seed được nạp. Các file database thiết bị, khóa ký, local.properties và cache build không được đưa vào Git.

CHƯƠNG 5. TRIỂN KHAI CHỨC NĂNG VÀ GIAO DIỆN

5.1. Thiết kế điều hướng và ngôn ngữ giao diện

Thiết kế giao diện dựa trên phong cách người dùng cung cấp: header tím gradient, các thẻ sáng bo góc, biểu đồ vòng nhiều màu, biểu đồ cột và thanh điều hướng dưới. Giao diện dùng tên chức năng tiếng Việt thay vì chỉ icon để người dùng dễ nhận biết. Icon được vẽ bằng Canvas nên không cần tải ảnh từ mạng.

ĐIỀU HƯỚNG MÀN HÌNH VÀ HỘP THOẠI



Điều hướng bằng trạng thái tab (0/1/2), không sử dụng NavController hoặc nhiều Activity.

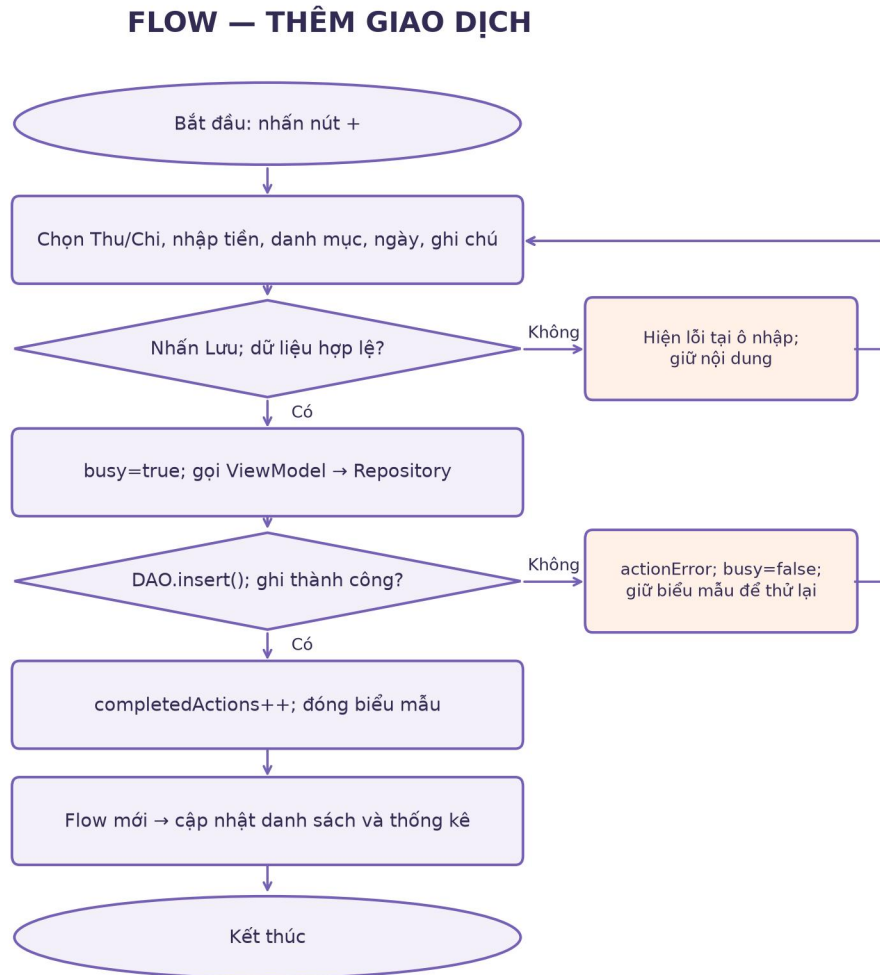
Hình 6. Điều hướng ba tab và các hộp thoại của QLCT

Bảng 27. Thành phần giao diện và tương tác

Thành phần	Dữ liệu hiển thị	Thao tác
DashboardHeader	Tháng/năm, chênh lệch, tổng thu, tổng chi	Nút trước/sau; chuyển Theo tháng/Toàn bộ.
CategoryChart	Vòng phân bổ, tên danh mục, %; số tiền chi tiết ở Thống kê	Xem dữ liệu theo period; chưa có drill-down khi chạm cung.
SpendingChart	Cột 6 tháng, nhãn tháng/năm; số VND chi tiết ở Thống kê	Đổi mốc bằng nút tháng; chưa có tooltip tương tác.
TransactionTile	Icon, tên, ngày, note rút gọn, tiền, thùng rác	Chạm dòng để sửa; thùng rác để yêu cầu xóa.
BottomNavigation	Tổng quan, Giao dịch, +, Thống kê	Chọn tab hoặc mở thêm giao dịch.
TransactionEditor	Thu/Chi, tiền, danh mục, ngày, note	Nhập/sửa và Lưu/Hủy; lỗi inline tại tiền.

5.2. Triển khai thêm giao dịch

Khi nút + được nhấn, editingId được đặt null và editorOpen=true. Điều kiện cho phép mở là dữ liệu đã tải, không loadError, danh mục không rỗng và không busy. TransactionEditor khởi tạo default từ existing hoặc giá trị mới. Trong trường hợp thêm, ID truyền vào TransactionEntity là 0 để Room sinh ID thay vì dùng ID do UI tự chọn.



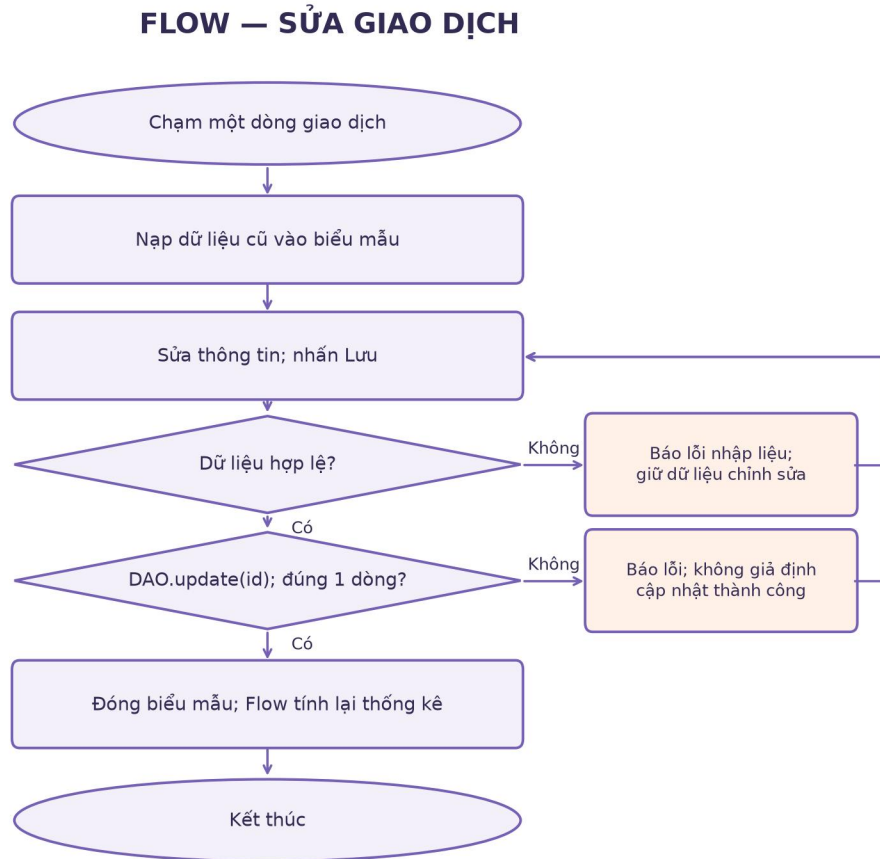
Hình 7. Flow thêm giao dịch với nhánh dữ liệu sai và ghi thất bại

Nhấn Lưu đặt amountChecked=true. Nếu AmountValidator.errorForInput trả lỗi, hệ thống đánh dấu isError ở OutlinedTextField và hiện supportingText cụ thể. Biểu mẫu không đóng và không gọi model.save(). Khi tiền hợp lệ nhưng danh mục không còn trong categories, hiện thông báo chọn danh mục. Chỉ khi các kiểm tra UI đạt mới gửi TransactionEntity tới ViewModel.

mutate() kiểm tra busy để chặn gọi lặp, sau đó chạy action trong viewModelScope.launch. Repository thực hiện bước kiểm tra nghiệp vụ thứ hai. Nếu

insert thành công, completedActions tăng; finally đưa busy về false. LaunchedEffect ở ExpenseScreen đóng editor; Flow mới tính lại các vùng hiển thị. Nếu thao tác phát exception, actionError hiện thông báo tổng quát và dữ liệu nhập được giữ để thử lại.

5.3. Triển khai sửa giao dịch



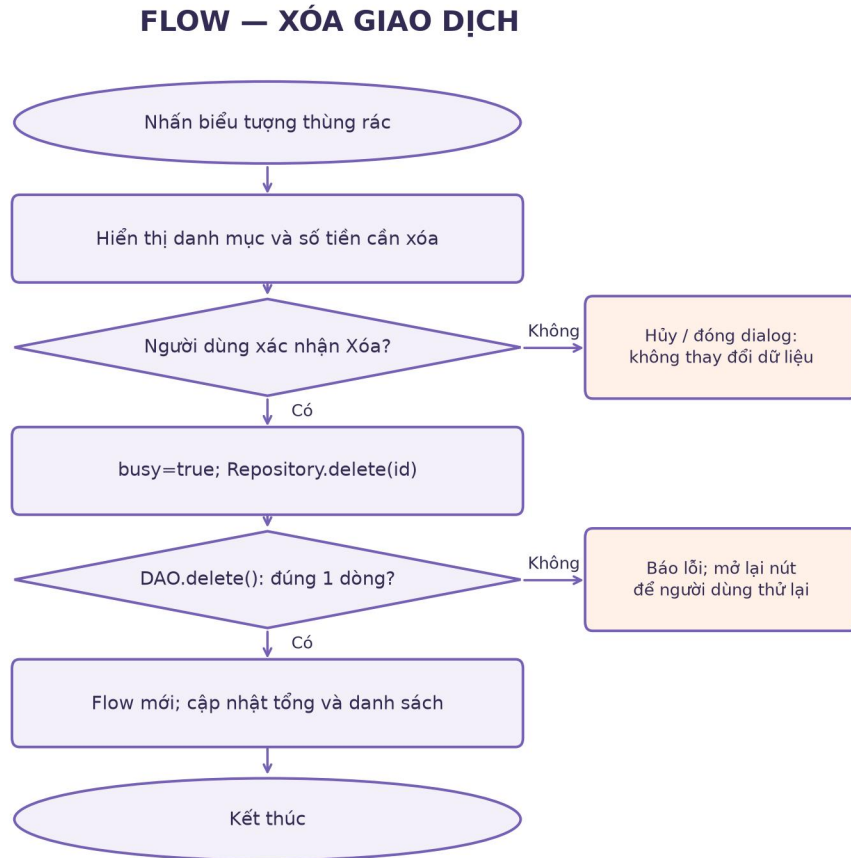
Hình 8. Flow sửa giao dịch và kiểm tra bản ghi còn tồn tại

Chạm dòng lưu editingId rồi tìm giao dịch trong state.summary.visible. Cùng TransactionEditor được sử dụng cho thêm và sửa để tránh hai bộ validate khác nhau. Khi existing không null, biểu mẫu nạp ID và các trường cũ. Lưu tạo entity với đúng ID cũ; repository gọi update thay vì insert và kiểm tra số dòng trả về bằng 1.

Người dùng có thể chuyển một khoản chi thành thu hoặc đổi ngày sang tháng khác. Tổng chi và tổng thu sẽ thay đổi tương ứng; dòng giao dịch có thể không còn xuất hiện do bộ lọc đang chọn. Hành vi này được xác định bằng dữ liệu sau update, không phải

lỗi mất dữ liệu. Khi người dùng hủy, những thay đổi chưa lưu chỉ tồn tại ở state nhập và không update Room.

5.4. Triển khai xóa có xác nhận

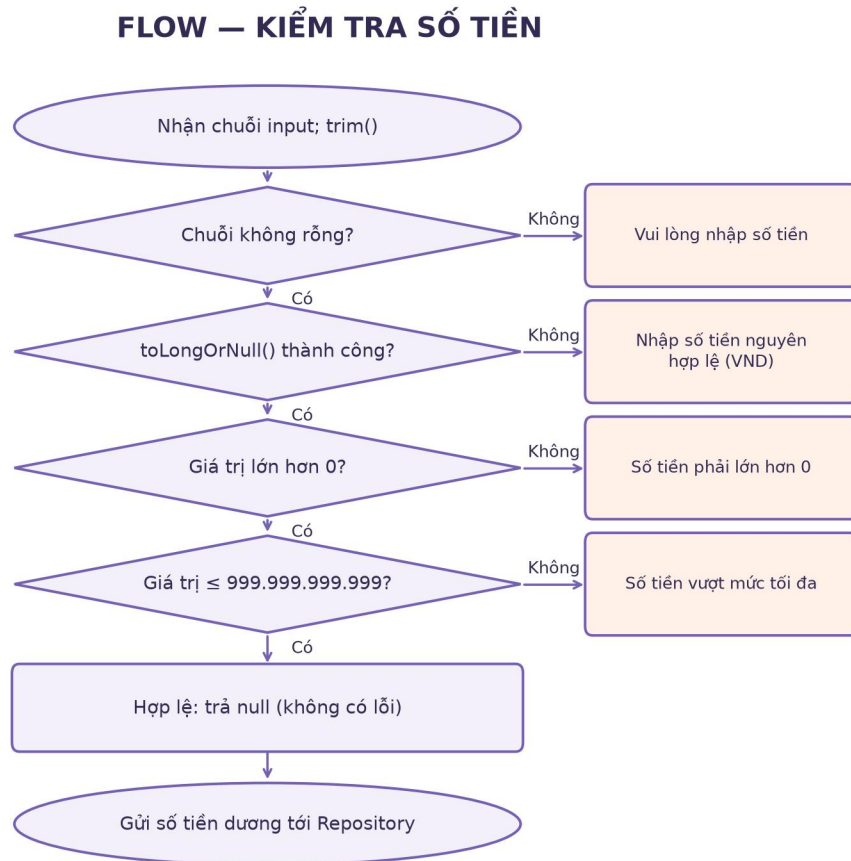


Hình 9. Flow xóa với nhánh hủy và lỗi ghi

Nút thùng rác tách biệt thao tác xóa với chạm sửa. Dialog hiển thị danh mục và tiền đề người dùng nhận diện bản ghi. Hủy chỉ đóng dialog. Xóa mới gọi `ViewModel.delete(transaction)`, lấy `transaction.id` và thực thi DAO query có tham số. Repository dùng check để bảo đảm một bản ghi được tác động.

Khi xóa thành công, `completedActions` khiến dialog đóng. Flow loại giao dịch khỏi kết quả đọc và summarize tính lại tổng, donut và cột. Không giảm tổng tiền bằng cách trừ trực tiếp trong `onClick`, nên các trường hợp khác thời gian hoặc đổi lọc vẫn theo cùng thuật toán. Hiện chưa có Undo hoặc thùng rác khôi phục; dialog đã thông báo tính không thể hoàn tác.

5.5. Validate số tiền và bảo vệ đầu vào



Hình 10. Flow validate số tiền từ chuỗi nhập tới Long hợp lệ

Bảng 28. Thông báo validate theo dữ liệu đầu vào

Đầu vào	Kết quả	Thông báo / hành vi
Rỗng hoặc chỉ khoảng trắng	Không hợp lệ	Vui lòng nhập số tiền.
abc; 1.5; 1,5; chuỗi vượt Long	Không hợp lệ	Nhập số tiền nguyên hợp lệ (VND).
-1; -50000; 0; -0	Không hợp lệ	Số tiền phải lớn hơn 0.
1000000000000	Không hợp lệ	Số tiền tối đa là 999.999.999.999 đ.
1; 50000; 999999999999	Hợp lệ	Không có lỗi; có thể lưu nếu các trường khác hợp lệ.
Khoảng trắng quanh 50000	Hợp lệ sau trim	Chuỗi được trim trước parse.

```

fun errorForValue(value: Long): String? = when {
    value <= 0 -> "Số tiền phải lớn hơn 0"
}
  
```

```

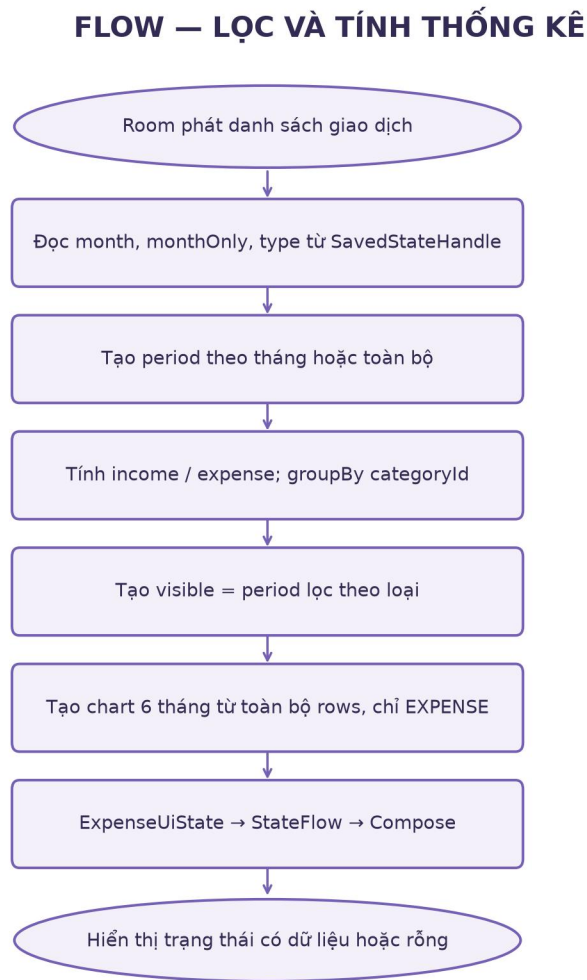
    value > MAX_AMOUNT -> "Số tiền tối đa là 999.999.999.999 đ"
    else -> null
}

```

Không đổi tiền âm thành trị tuyệt đối, vì cách đó có thể che giấu việc nhập sai. Người dùng nhập giá trị dương rồi chọn Thu hoặc Chi. Ví dụ chi 50.000 VND phải nhập 50000 và chọn Chi tiền. Giao diện hiển thị “–50.000 đ”, nhưng database giữ amount=50000 và type=EXPENSE. Quy tắc này giúp công thức sumOf đơn giản và tránh trường hợp một số âm bị trừ hai lần.

Bàn phím số được yêu cầu bằng KeyboardOptions(keyboardType=Number), nhưng keyboardType không thay thế validate: bàn phím vật lý, paste hoặc một IME khác có thể cung cấp ký tự không mong muốn. Vì vậy vẫn parse và kiểm tra trước khi lưu. Khi người dùng đã nhấn Lưu và tiếp tục sửa amount, lỗi được tính lại theo chuỗi hiện tại, không bắt buộc đóng/mở dialog.

5.6. Tổng hợp, lọc và các công thức thống kê



Hình 11. Flow tính period, visible, donut và chart

5.6.1. Xác định period và danh sách visible

```

period = rows.filter { !monthOnly || it.date.startsWith(month + "-") }
visible = period.filter { type == "ALL" || it.type == type }
  
```

Tiền được tổng hợp từ period trước bước type filter. Nhờ đó khi chọn Thu, danh sách chỉ hiện khoản thu nhưng thẻ Tổng chi vẫn phản ánh chi thực của kỳ. Trên tab Tổng quan, giao dịch gần đây dùng visible.take(4), vì vậy cũng tôn trọng loại lọc đang giữ từ tab Giao dịch. Nếu muốn xem toàn bộ dòng, chọn Tất cả.

5.6.2. Tổng thu, tổng chi và chênh lệch

```

income = Σ amount của t ∈ period có t.type = INCOME
expense = Σ amount của t ∈ period có t.type = EXPENSE
difference = income - expense
  
```

Với dữ liệu mẫu: $\text{income} = 3.000.000 + 1.200.000 = 4.200.000$;
 $\text{expense} = 45.000 + 70.000 + 150.000 + 250.000 + 800.000 + 30.000 = 1.345.000$;
 $\text{difference} = 2.855.000$. Nếu tổng chi lớn hơn tổng thu, chênh lệch âm được hiển thị hợp lệ. Ứng dụng chưa biết số dư tiền mặt ban đầu hoặc tài sản khác, nên chênh lệch này không được gọi là số dư toàn bộ tài sản.

5.6.3. Biểu đồ vòng

```
categoryAmount[c] =  $\Sigma$  amount của EXPENSE trong period có categoryId = c
ratio[c] = categoryAmount[c] / expense
sweepAngle[c] = ratio[c]  $\times$  360°
percentLabel[c] = floor(ratio[c]  $\times$  100)
```

Các nhóm được sắp giảm dần theo amount. Vòng vẽ đủ tỷ lệ từ số tiền thực; nhãn phần trăm dùng số nguyên làm tròn xuống. Với dữ liệu mẫu, phần nhãn có thể là 59%, 18%, 11%, 5%, 5% nên tổng nhãn là 98%; đây là hệ quả cách hiển thị chứ không mất tiền. Tab Thống kê có số tiền chính xác để đối chiếu. Một cải tiến có thể là hiển thị một chữ số thập phân hoặc phân bổ phần dư khi làm tròn.

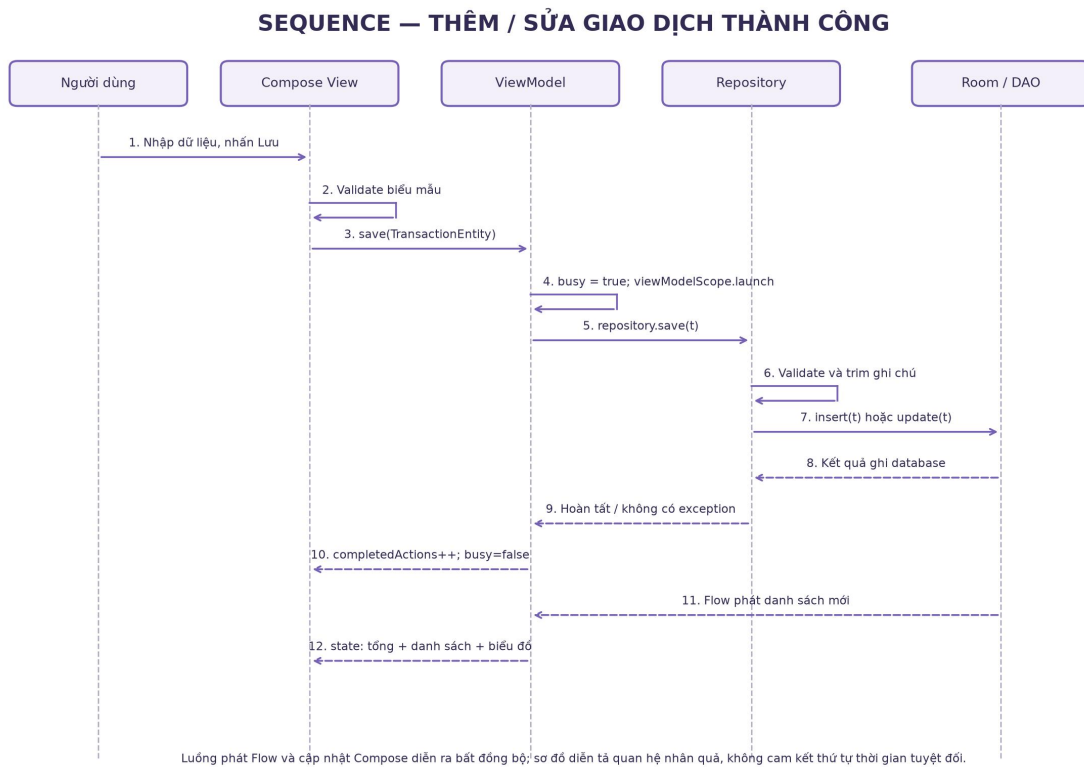
5.6.4. Biểu đồ cột sáu tháng

```
months = [M-5, M-4, M-3, M-2, M-1, M]
monthlyExpense[m] =  $\Sigma$  amount của rows có type=EXPENSE và tháng(date)=m
barHeight[m] = monthlyExpense[m] / max(1, maxMonthlyExpense)  $\times$  chartHeight
```

Biểu đồ cột lấy toàn bộ rows làm nguồn, không lấy visible hoặc chỉ period. Nếu M là 10/2026, sáu tháng là 5/2026 đến 10/2026. Đổi sang Toàn bộ chỉ ảnh hưởng period của tổng quan/donut/danh sách; chart vẫn là sáu tháng kết thúc tại M. Cách tách này phải được giải thích khi bảo vệ để tránh nhận định sai rằng bộ lọc đang gây lỗi biểu đồ.

Các phép chia tỷ lệ dùng Double/Float ở bước vẽ để chuẩn hóa độ cao/góc; tổng tiền vẫn giữ Long. Khi tất cả số tiền bằng 0, max được giữ ít nhất 1 để tránh phép chia 0. Với số lượng giao dịch cực lớn, phép sumOf Long vẫn có nguy cơ overflow; ứng dụng hiện chưa có kiểm tra tràn tổng và chưa có benchmark dữ liệu lớn.

5.7. Sequence ghi dữ liệu và cập nhật UI



Hình 12. Sequence save từ View tới Room và chiều cập nhật Flow

Sequence mô tả sự phối hợp giữa lớp, không phải giao tiếp mạng. Khi action hoàn tất và khi Flow phát giá trị mới là các sự kiện bất đồng bộ; không nên khẳng định cả hai xảy ra theo thứ tự thời gian tuyệt đối như một lời gọi đồng bộ duy nhất. Chỉ đóng dialog sau khi repository hoàn tất; dữ liệu hiển thị mới đến từ Room.

5.8. Xử lý trạng thái rỗng, lỗi và thao tác đồng thời

Loading ban đầu: hiển thị spinner. LoadError: thông báo không tải được dữ liệu và nút Thử lại. Danh sách không có giao dịch: hiển thị thẻ gợi ý nút +. Donut không có khoản chi: vòng nền và lời nhắc. Biểu đồ cột không có khoản chi: thông báo giai đoạn rỗng. Các trạng thái này có ý nghĩa khác nhau; lỗi database không được mô tả là người dùng chưa tạo giao dịch.

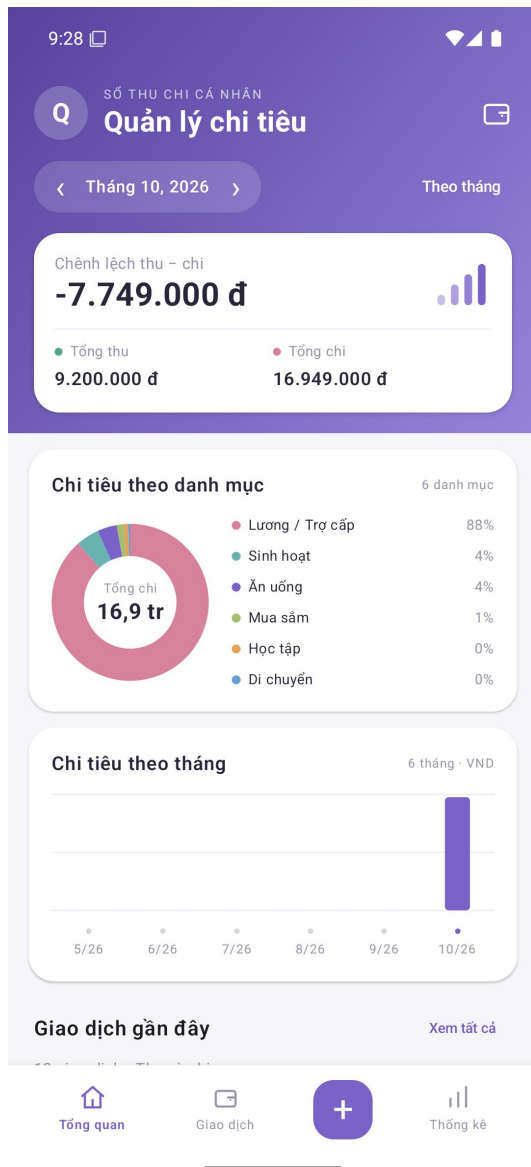
Trong khi ghi, busy ngăn nhiều yêu cầu save/delete chạy cùng lúc; các nút lưu/hủy hoặc hành động liên quan bị disable theo trạng thái. Khi thao tác thất bại, nội dung nhập vẫn ở rememberSaveable và busy được giải phóng trong finally. Thông báo lỗi ghi hiện mang tính tổng quát; bản hiện tại chưa có mã lỗi riêng cho disk full, database locked hoặc ID mất.

5.9. Ảnh chụp giao diện thực tế

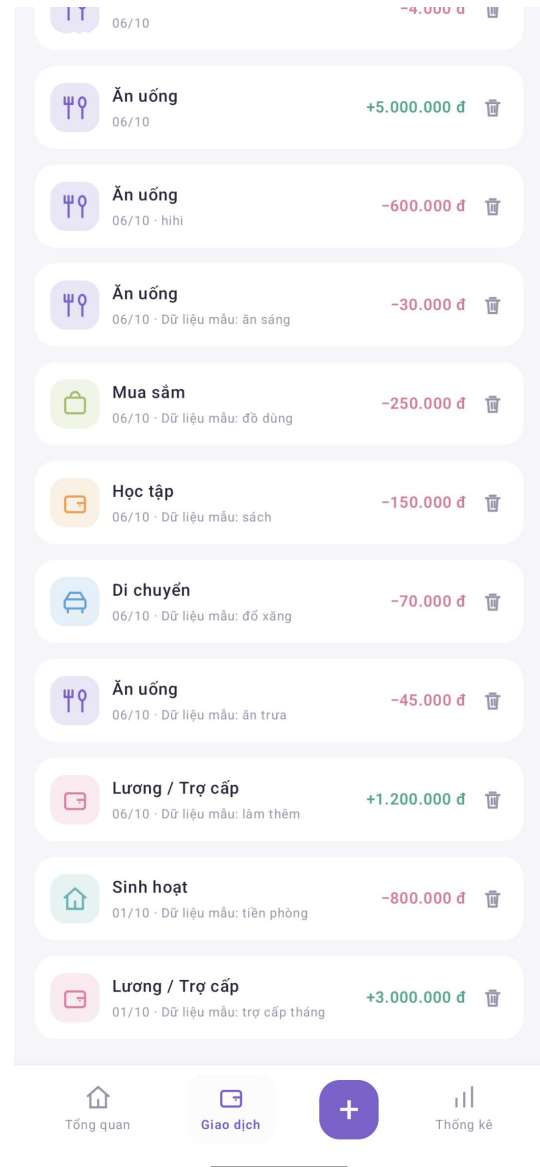
Các ảnh dưới đây được chụp từ máy ảo QLCT_API_35 đang chạy APK của project, không phải mockup. Giá trị giao dịch trong ảnh phản ánh dữ liệu của máy ảo tại thời điểm chụp. Ảnh validation là trạng thái đã kiểm tra nhập -50000; không có giao dịch âm được lưu.

5.9.1. Tổng quan và lịch sử

Ảnh chụp trực tiếp trên máy ảo Pixel 6 / Android 15 (API 35).



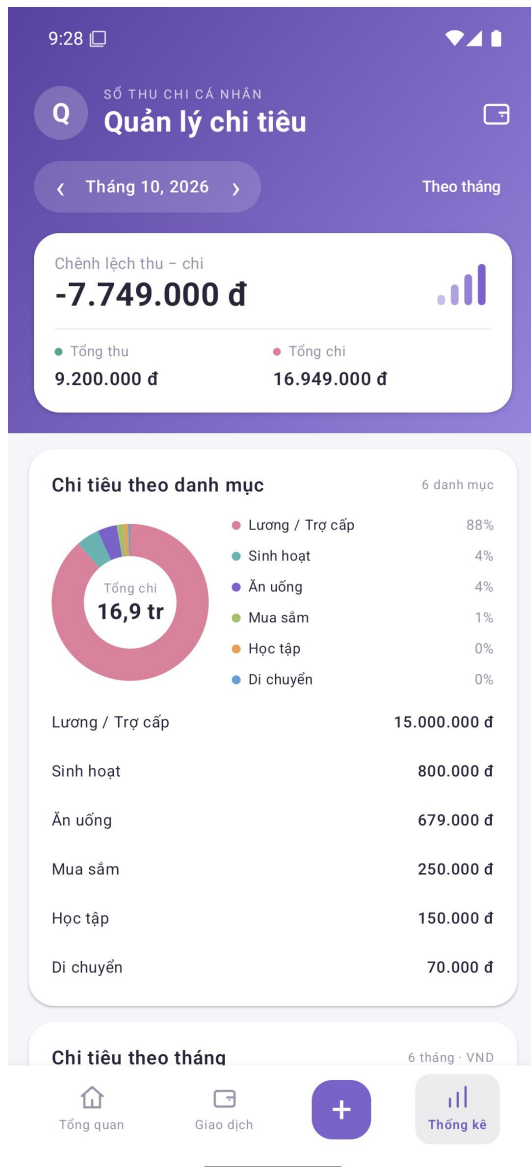
Hình 13. Màn hình Tổng quan với tổng thu/chi và hai biểu đồ



Hình 14. Màn hình Giao dịch, thứ tự ngày/ID và các thao tác

5.9.2. Thống kê và biểu mẫu thêm

Ảnh chụp trực tiếp trên máy ảo Pixel 6 / Android 15 (API 35).



Hình 15. Thống kê với số tiền theo danh mục

Quản lý chi tiêu

Tháng 10, 2026 Theo tháng

Chênh lệch thu - chi
-7.749.000 đ

Tổng thu: 9.200.000 đ Tổng chi: 16.949.000 đ

Thêm giao dịch

Chi tiền Thu tiền

Chi tiêu theo danh mục (6 danh mục)

Số tiền (VND)

Ngày: 06/10/2026

Lương / Trợ cấp: 15.000.000 đ

Sinh hoạt: 800.000 đ

Ăn uống: 679.000 đ

Mua sắm: 250.000 đ

Học tập: 150.000 đ

Di chuyển: 70.000 đ

Ghi chú (không bắt buộc)

Hủy Lưu

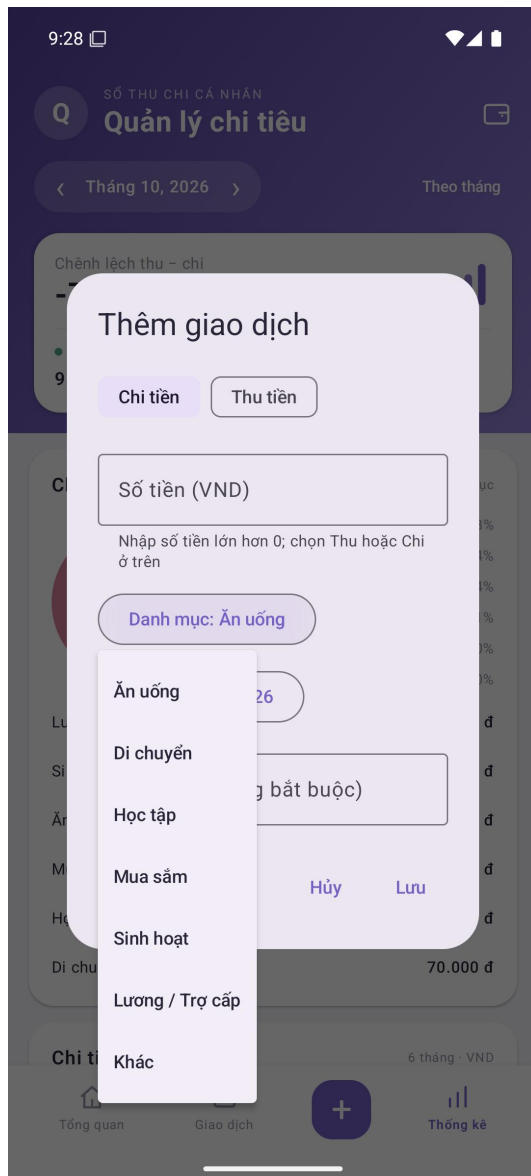
Chi tiêu theo tháng (6 tháng · VND)

Tổng quan Giao dịch **+** Thống kê

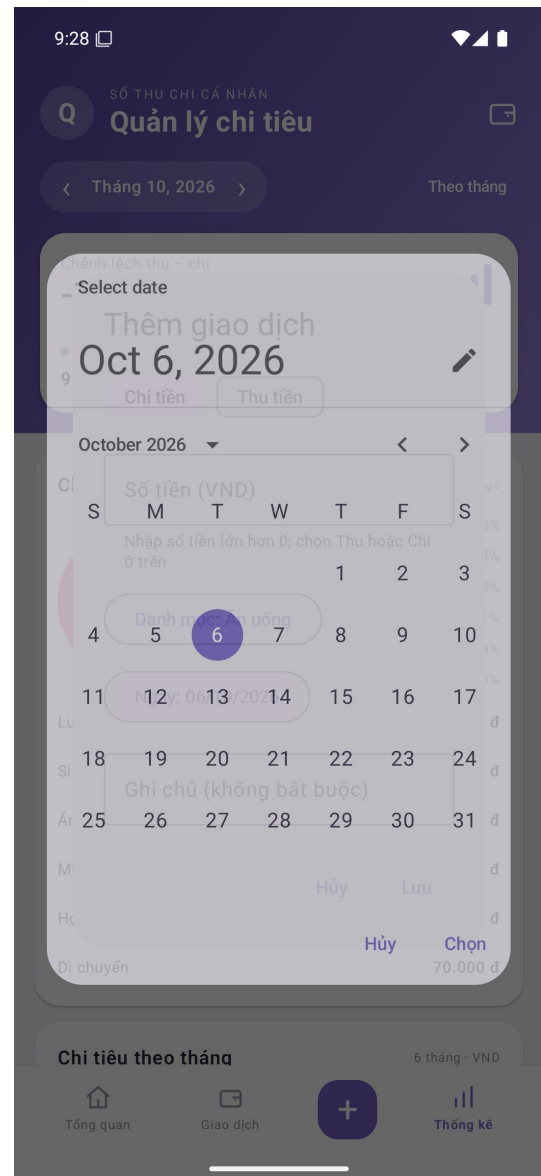
Hình 16. Biểu mẫu thêm giao dịch bằng Compose

5.9.3. Chọn danh mục và ngày

Ảnh chụp trực tiếp trên máy ảo Pixel 6 / Android 15 (API 35).



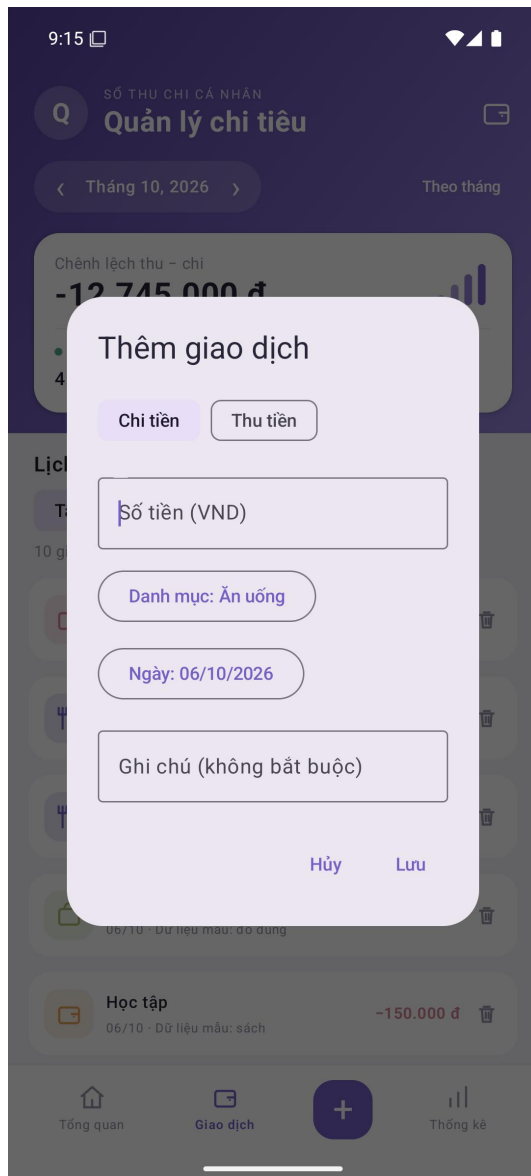
Hình 17. DropdownMenu chọn một trong 7 danh mục



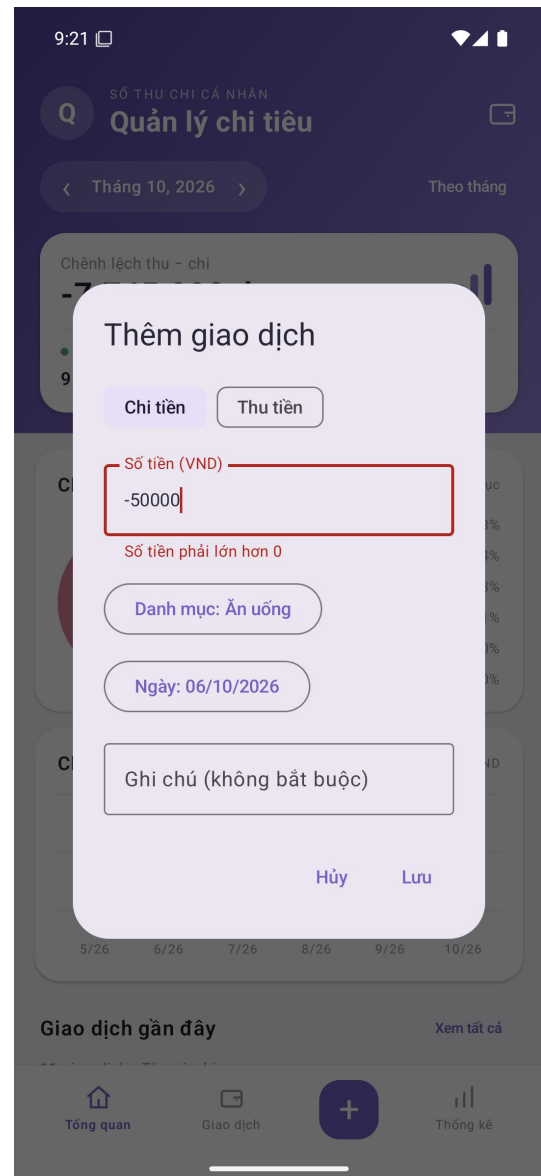
Hình 18. DatePickerDialog chọn ngày giao dịch

5.9.4. Nhập liệu và lỗi số tiền

Ảnh chụp trực tiếp trên máy ảo Pixel 6 / Android 15 (API 35).



Hình 19. Bàn phím số và ô nhập giao dịch trên máy ảo



Hình 20. Lỗi inline khi nhập số tiền âm, biểu mẫu được giữ

CHƯƠNG 6. KIỂM THỬ VÀ HƯỚNG DẪN VẬN HÀNH

6.1. Mục tiêu, phương pháp và môi trường

Kiểm thử tập trung vào tính đúng của số tiền, phạm vi thời gian, phân bổ danh mục, toàn vẹn dữ liệu và khả năng giữ dữ liệu khi chuyển từ SQLite cũ sang Room. Các lớp kiểm tra bổ sung cho nhau: unit test không cần thiết bị; instrumentation test dùng SQLite và Room thật trên Android; kiểm tra SQL host đối chiếu schema; thao tác UI xác nhận người dùng có thể nhập và nhận lỗi đúng. Không dùng kết quả của một lớp để khẳng định mọi tình huống ở lớp khác đều đã được kiểm tra.

Bảng 29. Môi trường kiểm thử và giới hạn bằng chứng

Hạng mục	Cấu hình / phạm vi
Máy phát triển	Windows; Android Studio; project D:/AndroidStudioWorkspace/QLCT_LTDD.
Thiết bị	AVD QLCT_API_35; Pixel 6; Android 15/API 35; Google APIs x86_64; RAM 4 GB.
Ứng dụng	APK debug, package com.example.qlct; database Room version 2.
Unit test	JUnit 4.13.2; 4 test AmountValidatorTest và 3 test ExpenseSummaryTest.
Instrumentation	ExpenseDatabaseTest: 1 phương thức kiểm thử tích hợp chứa nhiều kiểm tra migration, CRUD và validate.
SQL host	docs/tests/test_room_migration.py; so sánh cấu trúc với schema Room xuất bởi KSP.
Giới hạn	Chưa kiểm tra đầy đủ trên điện thoại thật, nhiều kích thước màn hình, xoay màn hình, process death và lỗi ổ đĩa.

6.2. Kết quả kiểm thử tự động

Bảng 30. Các unit test đã đạt

Mã	Test / mục tiêu	Kết quả
UT01	negativeAndZeroAmountsAreRejected: -1, -50000, 0, -0 và Long.MIN_VALUE.	Đạt
UT02	positiveBoundsAreAccepted: 1, 50000, trim khoảng trắng và MAX_AMOUNT.	Đạt
UT03	emptyDecimalsAndMalformedInputAreRejected: rỗng, số thập phân, chữ, dấu sai và tràn Long.	Đạt
UT04	amountsAboveLimitAreRejected: 1000000000000 và Long.MAX_VALUE.	Đạt
UT05	filtersDoNotChangeChartAndIncomeIsExcludedFromExpense: tổng thu/chi, lọc loại, sáu tháng qua năm mới.	Đạt
UT06	emptyMonthsAndLeapDayAreHandled: 29/02 năm nhuận, tháng rỗng và số tiền lớn trong giới hạn.	Đạt

Mã	Test / mục tiêu	Kết quả
UT07	categoryChartGroupsExpensesAndRespectsPeriodButNotTypeFilter: nhóm danh mục, thứ tự giảm dần, bỏ khoản thu.	Đạt

Kết quả 7/7 là số phương thức JUnit, không phải số tổ hợp dữ liệu. Một phương thức có thể chạy nhiều giá trị đầu vào. Các báo cáo máy đọc nằm trong app/build/test-results/testDebugUnitTest/*.xml. Test thống kê dùng dữ liệu giả được tạo trong bộ nhớ và kiểm tra giá trị cụ thể; không phụ thuộc giao dịch đang có trên máy ảo.

Bảng 31. Các kiểm tra trong instrumentation test

Nhóm	Điểm đối chiếu	Kết quả
Migration v1 → v2	Tạo database theo schema cũ, mở bằng Room với migration, dữ liệu danh mục/giao dịch và ID được giữ.	Đạt
Insert / Read	Thêm giao dịch hợp lệ; đọc được giá trị và tên danh mục qua DAO/Flow.	Đạt
Update	Sửa giao dịch cùng ID; kết quả mới thay thế thông tin cũ.	Đạt
Delete	Xóa giao dịch đã tạo trong database kiểm thử; không còn dòng đó.	Đạt
Persistence	Đóng và mở lại database kiểm thử; dữ liệu đã ghi vẫn còn.	Đạt
Database rỗng	Các truy vấn xử lý database rỗng theo hợp đồng.	Đạt
Repository validate	Insert và update từ chối tiền âm, 0 và vượt MAX_AMOUNT.	Đạt

Instrumentation đạt 1/1 phương thức. Database kiểm thử tách khỏi dữ liệu sử dụng bình thường; thao tác xóa trong test không có nghĩa là xóa lịch sử của người dùng. Test Room kiểm tra luồng dữ liệu thực trên Android, nhưng không tự động chạm tất cả nút Compose.

Kiểm tra migration bằng Python trên SQLite máy tính đã đạt: chạy SQL chuyển đổi, đối chiếu bảng, cột, chỉ mục, khóa ngoại và dữ liệu bảo toàn với schema Room v2. Kiểm tra SQLite Basic cũ là bằng chứng lịch sử bổ sung, không thay thế instrumentation của bản Room hiện tại. Bản build debug và APK kiểm thử đã tạo thành công; lint gần nhất có 0 lỗi và 15 cảnh báo, chủ yếu liên quan phiên bản dependency, backup rules và tài nguyên không dùng. Không mô tả kết quả lint là không còn cảnh báo.

6.3. Kiểm thử giao diện trực tiếp và ma trận nghiệm thu

Trên máy ảo đã kiểm tra nhập 123 bằng bàn phím số, nhập ghi chú đánh dấu kiểm thử, nhấn Lưu và xác nhận biểu mẫu đóng, giao dịch xuất hiện. Dòng kiểm thử này đã được xóa sau khi xác nhận; không dùng nó làm dữ liệu mẫu của đồ án. Với 0 và -

50000, nhấn Lưu hiển thị lỗi “Số tiền phải lớn hơn 0”, giữ biểu mẫu và không ghi giao dịch. Các ảnh chụp chương 5 là bằng chứng giao diện thực. Việc chụp tab hoặc dialog chỉ chứng minh màn hình mở được, không tự động chứng minh toàn bộ chức năng nghiệp vụ đã qua kiểm thử.

Bảng 32. Ma trận chức năng và tình trạng xác minh

Mã	Kịch bản / kết quả mong đợi	Tình trạng
TC01	Mở ứng dụng, tab Tổng quan hiển thị; đọc dữ liệu đã có.	Đã quan sát trên AVD
TC02	Thêm khoản chi 123; biểu mẫu đóng và dòng mới xuất hiện.	Đã kiểm tra UI
TC03	Nhập 0 hoặc -50000; lỗi inline, không lưu.	Đã kiểm tra UI + unit + repository
TC04	Nhập rỗng, chữ, thập phân và vượt giới hạn; bị từ chối.	Unit đạt; cần thử đủ UI
TC05	Sửa amount/note và giữ ID; biểu đồ/tổng cập nhật.	Repository đạt; cần nghiệm thu UI đầy đủ
TC06	Xóa dòng kiểm thử; dòng không còn.	Đã kiểm tra UI + repository
TC07	Bấm Hủy xóa; dữ liệu giữ nguyên.	Cần nghiệm thu UI
TC08	Lọc Thu/Chi; tổng kỳ không bị lọc loại làm sai.	Unit đạt; cần thử đủ UI
TC09	Đổi tháng qua năm mới; chart có sáu tháng chính xác.	Unit đạt; cần thử đủ UI
TC10	Donut nhóm chi theo danh mục, bỏ thu và tôn trọng kỳ.	Unit đạt; đã quan sát màn hình
TC11	Migration giữ dữ liệu v1 và mở lại database.	Instrumentation + SQL host đạt
TC12	Tạo giao dịch khi không có Internet.	Thiết kế local, không có INTERNET; cần demo offline
TC13	Xoay màn hình / process death khi nhập dữ.	Chưa kiểm thử đầy đủ
TC14	Giả lập lỗi đọc/ghi; Thử lại, giữ biểu mẫu và giải phóng busy.	Có nhánh xử lý; chưa kiểm thử lỗi cường bức
TC15	Dữ liệu rất lớn, thiết bị thật và Dark Mode.	Chưa benchmark / chưa nghiệm thu đầy đủ

6.4. Cách tái thực hiện các kiểm tra

1. Mở project bằng Android Studio, chờ Gradle Sync hoàn tất. Kiểm tra SDK theo local.properties và phiên bản JDK/Gradle của project.
2. Chạy unit test và lint bằng lệnh dưới đây; đọc báo cáo XML/HTML trong app/build thay vì chỉ nhìn trạng thái IDE.
3. Bật QLCT_API_35 trong Device Manager, đợi hệ thống Android khởi động xong; chạy connectedDebugAndroidTest khi thiết bị đã được ADB nhận.

4. Chạy script migration host sau khi build đã xuất schema KSP. Nếu schema chưa có, thực hiện assembleDebug trước.
5. Đối chiếu số test, failures/errors và thời điểm chạy. Không đưa kết quả của build cũ vào báo cáo sau khi sửa nghiệp vụ mà chưa chạy lại kiểm tra liên quan.

```
.\gradlew.bat assembleDebug
.\gradlew.bat testDebugUnitTest lintDebug
.\gradlew.bat connectedDebugAndroidTest
python docs/tests/test_room_migration.py
```

Các lệnh được chạy tại thư mục gốc project bằng PowerShell. connectedDebugAndroidTest yêu cầu thiết bị đang chạy; các lệnh còn lại không cần mở app. Thư mục build là sản phẩm sinh tự động và có thể không được commit; vì vậy máy khác cần chạy lại để tạo bằng chứng mới.

6.5. Cài đặt và sử dụng ứng dụng

1. Trong Android Studio, chọn cấu hình app và máy ảo QLCT_API_35, nhấn Run. API 35 đáp ứng minSdk 24 dù target/compile SDK của project là 37.
2. Lần tạo database mới, app nạp 7 danh mục và 8 giao dịch mẫu theo ngày hiện tại. Cài đề APK giữ database; gỡ app hoặc xóa dữ liệu app sẽ tạo lại dữ liệu khi chạy lần tiếp theo.
3. Chạm + để thêm. Chọn Thu/Chi, nhập số nguyên dương không có dấu phân cách, chọn danh mục/ngày, thêm ghi chú nếu cần, nhấn Lưu.
4. Vào Giao dịch để xem danh sách và dùng hành động sửa/xóa ở mỗi dòng. Xóa cần xác nhận; Hủy giữ dữ liệu.
5. Dùng điều khiển tháng và Tháng này/Toàn bộ để chọn phạm vi tổng hợp; dùng Tất cả/Thu/Chi để lọc danh sách.
6. Vào Thống kê để đọc tiền chi từng danh mục. Biểu đồ cột luôn là sáu tháng kết thúc tại tháng đang chọn.

6.6. Lưu ý nhập liệu trên máy ảo

AVD đã cấu hình hw.keyboard=yes, hw.keyboard.lid=no và hiển thị IME với bàn phím vật lý. Người dùng có thể gõ từ bàn phím PC hoặc chạm các phím số Android. Nếu chỉ thấy thanh IME nhỏ, kiểm tra cài đặt bàn phím ảo và cold boot sau thay đổi cấu hình AVD. Đây là cấu hình môi trường, không phải lỗi StateFlow hay Room. Giá trị keyboardType=Number chỉ gợi ý bố cục; validate vẫn bắt buộc với nội dung paste và bàn phím vật lý.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết quả đạt được

Đồ án triển khai được chu trình ghi nhận thu/chi, xem, sửa và xóa giao dịch; chọn thời gian; lọc loại; tính tổng; phân tích chi theo danh mục và sáu tháng. Giao diện Compose chia thành ba tab với phong cách tím thống nhất. MVVM, StateFlow và Repository tạo luồng dữ liệu rõ ràng; Room cung cấp mapping, DAO và migration từ database cũ. Tiền dùng Long và validator chung giữa UI/Repository, chặn số âm, 0, chuỗi sai và vượt giới hạn.

Bảng chứng hiện có xác nhận 7 unit test, 1 instrumentation test tích hợp, kiểm tra SQL migration, build và một số thao tác UI trên AVD. Báo cáo mô tả cả nhánh bình thường, ngoại lệ, trạng thái rỗng và giới hạn. Mức hoàn thiện phù hợp phạm vi ứng dụng quản lý chi tiêu cá nhân cục bộ của đồ án; các chức năng không có trong mã không được tính vào kết quả triển khai.

7.2. Hạn chế hiện tại

- Danh mục cố định; chưa có CRUD danh mục, ngân sách, tìm kiếm nội dung và lọc theo danh mục.
- Chưa có tài khoản, đồng bộ đám mây, xuất Excel/PDF hoặc nhập dữ liệu; không có undo sau xóa.
- DAO tải toàn bộ giao dịch, summarize chạy trên tập dữ liệu trong bộ nhớ; chưa phân trang, benchmark hoặc chống overflow tổng Long.
- Các nhãn phần trăm donut làm tròn xuống; tổng nhãn có thể chưa bằng 100%.
- Dữ liệu tài chính chưa mã hóa; cấu hình backup còn điểm cần hoàn thiện; không khẳng định đây là giải pháp bảo mật tài chính ở mức thương mại.
- Chưa kiểm thử đầy đủ thiết bị thật, accessibility, dark theme, xoay màn hình, process death và lỗi cưỡng bức.

7.3. Kế hoạch phát triển theo mức ưu tiên

Bảng 33. Đề xuất phát triển sau phạm vi hiện tại

Ưu tiên	Nội dung	Điều kiện nghiệm thu đề xuất
1	Hoàn thiện nghiệm thu UI và accessibility; thêm Compose UI tests.	TC04–TC15 được thực hiện, kết quả và ảnh bằng chứng được lưu.
2	Tìm kiếm, lọc danh mục, CRUD danh mục.	Quy định xử lý danh mục đã được giao dịch tham chiếu; có migration mới.

Ưu tiên	Nội dung	Điều kiện nghiệm thu đề xuất
3	Ngân sách tháng và cảnh báo vượt mức.	Xác định rõ ngưỡng, kỳ và danh mục; unit test biên tháng.
4	Export/import, backup do người dùng điều khiển.	Có preview trước import, validate và chống ghi trùng.
5	Tối ưu truy vấn tổng hợp, phân trang và chống tràn tổng.	Đo với dữ liệu lớn, so sánh độ đúng và thời gian đáp ứng.
6	Tài khoản/đồng bộ nếu yêu cầu thay đổi.	Thiết kế xác thực, quyền truy cập, xung đột và bảo vệ dữ liệu trước triển khai.

Bảng trên là hướng phát triển, không phải danh sách tính năng đã hoàn thành. Mỗi thay đổi database cần tăng version, bổ sung migration và kiểm tra dữ liệu cũ; không dùng destructive migration để tiện thử nghiệm trên dữ liệu thật.

TÀI LIỆU THAM KHẢO

- [1] Android Developers — Jetpack Compose documentation. <https://developer.android.com/develop/ui/compose/documentation>. Truy cập ngày 06/10/2026.
- [2] Android Developers — Guide to app architecture. <https://developer.android.com/topic/architecture>. Truy cập ngày 06/10/2026.
- [3] Android Developers — ViewModel overview. <https://developer.android.com/topic/libraries/architecture/viewmodel>. Truy cập ngày 06/10/2026.
- [4] Android Developers — Save data in a local database using Room. <https://developer.android.com/training/data-storage/room>. Truy cập ngày 06/10/2026.
- [5] Android Developers — Migrate your Room database. <https://developer.android.com/training/data-storage/room/migrating-db-versions>. Truy cập ngày 06/10/2026.
- [6] Mã nguồn đồ án QLCT_LTDD: app/src/main, app/src/test, app/src/androidTest, app/schemas và docs/tests. Phiên bản đối chiếu ngày 06/10/2026. Các mô tả chức năng và cấu trúc SQL trong báo cáo được đối chiếu trực tiếp với repository.

PHỤ LỤC A. ẢNH XẠ MÃ NGUỒN VÀ TÀI LIỆU

Bảng 34. Tập phục vụ đọc và bảo vệ đồ án

Tệp / thư mục	Vai trò
app/src/main/java/com/example/qlct/MainActivity.kt	Điểm vào Activity và thiết lập giao diện.
app/src/main/java/com/example/qlct/ui/ExpenseScreen.kt	Ba tab, editor, dialog xóa, điều khiển lọc và vẽ biểu đồ.
app/src/main/java/com/example/qlct/ui/ExpenseViewModel.kt	UiState, StateFlow, SavedStateHandle, save/delete/retry và summarize.
app/src/main/java/com/example/qlct/data/	Entities, DAO, ExpenseDatabase, ExpenseRepository và AmountValidator.
app/src/main/assets/seed.sql	7 danh mục và 8 giao dịch mẫu, nạp khi tạo database mới.
app/schemas/	Schema Room xuất để tham chiếu và kiểm tra migration.
app/src/test/ và app/src/androidTest/	Unit tests và kiểm thử tích hợp Android.
docs/tests/	Kiểm tra SQL migration trên máy tính.
docs/report_diagrams.py	Nguồn sinh 12 sơ đồ; PNG/SVG trong docs/diagrams/current/.
docs/build_final_report.py và report_sections_*.py	Nguồn biên soạn Word; nội dung và hình có thể chỉnh rồi sinh lại.
docs/screenshots/	Ảnh thật từ máy ảo phục vụ minh họa báo cáo.

Các đường dẫn là tương đối từ thư mục gốc project. Thư mục cache .gradle/report-libs chứa thư viện Python phục vụ tạo báo cáo, không phải dependency Android. Báo cáo Basic cũ được giữ như tài liệu lịch sử; báo cáo hiện tại là Bao_cao_Do_an_QLCT_Compose_MVVM_Room.docx.

PHỤ LỤC B. KỊCH BẢN THUYẾT TRÌNH VÀ DEMO

Kịch bản này là hướng dẫn đề xuất cho buổi bảo vệ, chưa được tính là kết quả kiểm thử đã chạy. Có thể dùng dữ liệu hiện tại; nếu dùng tổng mẫu phải xác nhận database thật đang có đúng 8 dòng seed. Không xóa dữ liệu cá nhân để khớp con số trong báo cáo.

Bảng 35. Kịch bản demo có thể thực hiện

Bước	Thao tác	Điểm trình bày
1	Giới thiệu mục tiêu và thành viên.	Ứng dụng Android cục bộ, thu/chi, Compose–MVVM–Room.
2	Mở Tổng quan; giải thích các thẻ tổng.	Chênh lệch = thu – chi; không phải toàn bộ tài sản.
3	Nhấn +; chọn Chi, nhập -50000, nhấn Lưu.	Lỗi inline và biểu mẫu giữ nguyên; database không bị ghi.
4	Sửa thành 50000; chọn danh mục và ngày; ghi chú DEMO; Lưu.	Tiền dương, loại quyết định thu/chi; dữ liệu lưu Room.
5	Vào Giao dịch, tìm dòng DEMO theo vị trí ngày.	Dòng sắp theo date DESC rồi id DESC.
6	Sửa dòng DEMO thành 60000 và lưu.	Giữ ID, các tổng tự cập nhật từ Flow.
7	Chọn Thu, Chi và Tất cả.	Danh sách thay đổi; tổng kỳ không phụ thuộc lọc loại.
8	Đổi tháng và bật Toàn bộ.	Giải thích period và chart sáu tháng độc lập.
9	Vào Thống kê.	Danh mục chỉ cộng chi, số tiền chính xác và phần trăm làm tròn.
10	Xóa dòng DEMO, xác nhận.	Dữ liệu và thống kê cập nhật; không có undo.
11	Mở ERD và sequence trong báo cáo.	Liên hệ FK, validator, ViewModel và DAO.
12	Trình bày test và hạn chế.	Phân biệt đã kiểm tra và cần kiểm tra; kế hoạch phát triển.

Nếu demo chọn một ngày ngoài tháng đang xem, giao dịch vẫn được lưu nhưng có thể không xuất hiện trong danh sách của tháng hiện tại. Chuyển sang tháng tương ứng hoặc Toàn bộ để kiểm tra. Đây là hệ quả bộ lọc, không nên kết luận thao tác Lưu thất bại chỉ vì chưa thấy dòng.

PHỤ LỤC C. CÂU HỎI BẢO VỆ VÀ GIẢI THÍCH KỸ THUẬT

C.1. Vì sao tiền phải dương trong cả thu và chi?

type đã biểu diễn hướng giao dịch. Lưu số dương giúp sum rõ nghĩa, tránh trừ số âm hai lần và phát hiện nhập nhầm. Chênh lệch tính ra vẫn có thể âm hợp lệ.

C.2. Vì sao dùng Long thay vì Double?

VND trong phạm vi đồ án dùng số nguyên; Long không có sai số dấu phẩy động khi cộng tiền. Double/Float chỉ dùng ở bước tỷ lệ vẽ. Tổng Long vẫn cần bảo vệ overflow khi mở rộng quy mô.

C.3. Room khác SQLite như thế nào?

SQLite là hệ quản trị dữ liệu nằm trên thiết bị; Room là lớp mapping/DAO trên SQLite, kiểm tra truy vấn và schema, tích hợp Flow và migration. Room không thay SQLite bằng database trên server.

C.4. MVVM phân công trách nhiệm ra sao?

View nhận thao tác và hiển thị; ViewModel giữ trạng thái/lọc và phối hợp action; Repository kiểm tra nghiệp vụ và truy cập DAO; Room thực hiện lưu trữ. Composable không viết SQL trực tiếp.

C.5. Tại sao Flow làm giao diện cập nhật?

Room phát lại kết quả khi bảng liên quan thay đổi; ViewModel combine dữ liệu với lựa chọn lọc thành StateFlow; Compose thu nhận state và tái tạo phần giao diện cần cập nhật.

C.6. Dữ liệu có mất khi cài bản APK mới không?

Cài đề cùng package giữ dữ liệu. Database v1 có migration sang v2. Gỡ app hoặc clear data làm mất dữ liệu ứng dụng. Chưa có backup/export do người dùng điều khiển.

C.7. Tại sao không dùng destructive migration?

Nó có thể xóa dữ liệu khi thiếu đường chuyển đổi. Đồ án có dữ liệu thu/chi cần giữ nên migration tạo cấu trúc mới và khôi phục dòng/ID; version khác chưa có migration phải xử lý rõ.

C.8. Tại sao chart không đổi khi lọc Thu?

Bộ lọc loại áp dụng visible; chart lấy EXPENSE từ toàn bộ rows trong sáu tháng. Tổng kỳ và donut cũng được tính trước lọc loại, để không mất thông tin chi khi xem danh sách thu.

C.9. Tổng phần trăm donut vì sao có thể là 98%?

Mỗi nhãn bị làm tròn xuống thành số nguyên. Góc vẽ dựa trên tỷ lệ thực nên không thiếu dữ liệu. Có thể nâng cấp cách làm tròn hoặc hiển thị phần thập phân.

C.10. Foreign key và index có ý nghĩa gì?

category_id tham chiếu categories.id để tránh giao dịch mờ mờ; NO ACTION không cho xóa danh mục còn được tham chiếu. Index ngày và category_id hỗ trợ truy vấn/khóa ngoại; không đảm bảo mọi truy vấn JOIN đều không cần sort.

C.11. Có bảo vệ số âm ngay trong SQL không?

Schema Room v2 hiện không có CHECK cho amount/type. UI và Repository kiểm tra; FK/NOT NULL/UNIQUE là ràng buộc database. Gọi DAO trực tiếp có thể bỏ qua validator nên cần duy trì tầng Repository và cân nhắc CHECK ở nâng cấp sau.

C.12. Đã kiểm thử mọi màn hình chưa?

Chưa. Có 7 unit test và 1 instrumentation tích hợp, kiểm tra host và một số thao tác UI thật. Các case xoay/process death/lỗi cưỡng bức/thiết bị thật được ghi là chưa nghiệm thu, không ghi thành đạt.

PHỤ LỤC D. CHECKLIST TRƯỚC KHI NỘP

- Bổ sung đúng tên trường, khoa, lớp, giảng viên và năm học ở trang bìa; xác nhận bốn họ tên/mã sinh viên.
- Đối chiếu mẫu trình bày của trường; chỉnh lề/font nếu có quy định riêng. Không tự ghi tên giảng viên chưa được xác nhận.
- Xác nhận phân công nhóm thực tế; thay phương án đề xuất bằng nội dung được các thành viên thống nhất.
- Sau khi sửa nội dung, chọn Ctrl+A rồi F9 trong Word để cập nhật mục lục, danh mục hình/bảng và số trang.
- Nghiệm thu các case đang đánh dấu cần kiểm tra; chỉ đổi thành Đạt khi có thao tác/kết quả thực.
- Chuẩn bị project mở được, APK debug và máy ảo; tập demo thêm/sửa/xóa bằng dữ liệu đánh dấu riêng.
- Không nộp local.properties, khóa ký, database riêng hoặc cache như một phần mã nguồn; tham khảo quy định nộp bài của môn.
- Đọc lại giới hạn và tài liệu tham khảo; kiểm tra hình/sơ đồ rõ nét trong cả Word và PDF.